UI BUILDER

AUTOMATIC CODE GENERATION TOOL TO DESIGN AND DEVELOP UI

Project ID: 2022-070



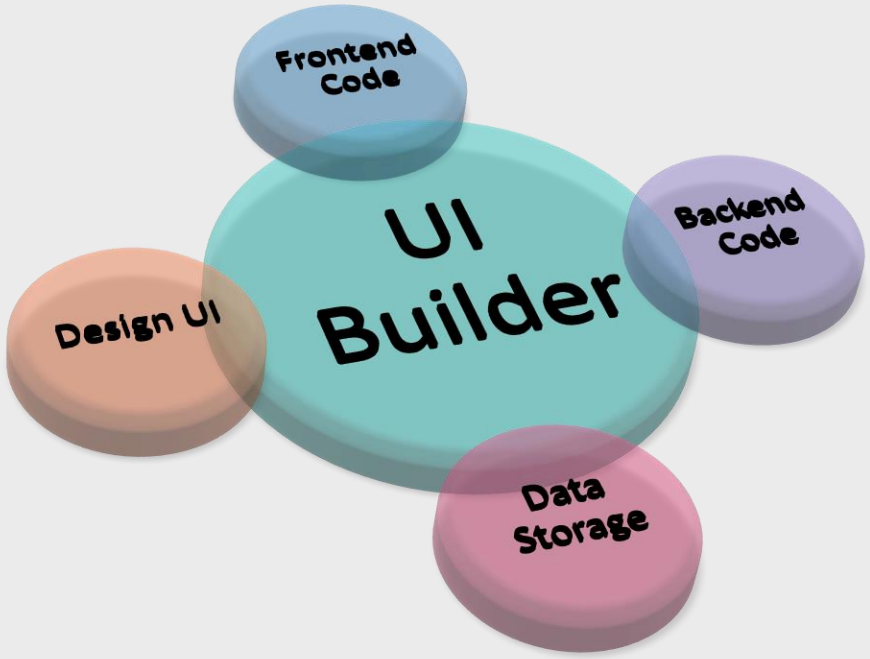
SLIIT
FACULTY OF COMPUTING

TEAM MEMBERS

Group ID: 2022-070	
Supervisor: Dr. Nuwan Kodagoda	
Co Supervisor: Ms. Kushnara Suriyawansa	
Student Name	**Student ID**
Ekanayake K.L.W (Group Leader)	IT19150758
Sakthivel.S	IT19158228
H.M. Kasuni Navodya	IT19144986
D.P Kavindi Gimshani	IT19150826

INTRODUCTION

- What is Automatic Code generation (ACG)?
- What is the impact of ACG on software development process?
- Why do we need a new tool for ACG?



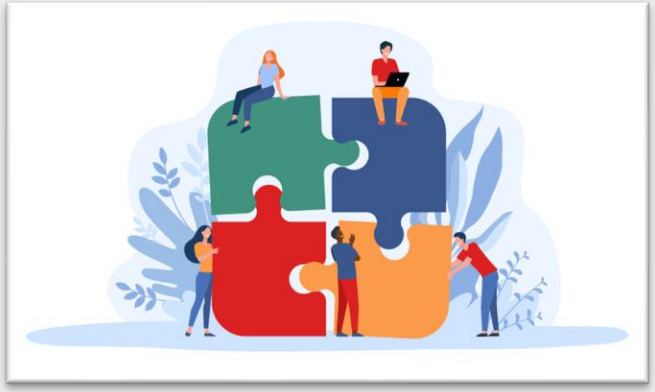
RESEARCH PROBLEM



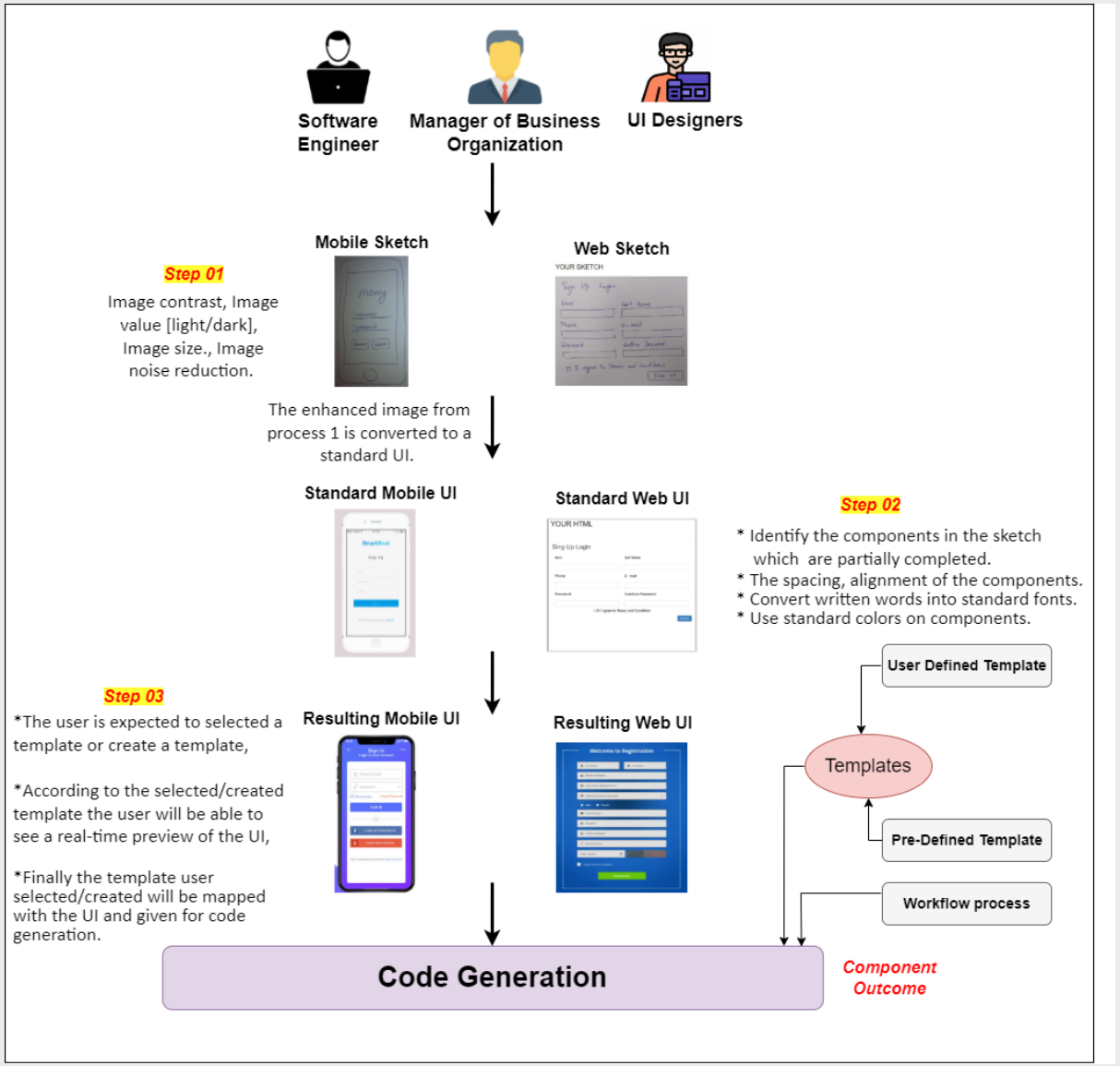
1. Need to follow the traditional method of software engineering process.
2. Available tools don’t fulfill the real requirements of software developers.
3. Auto-generated Code do not have the texture of real-handwritten code.
4. Auto-generated code only focuses on frontend, whereas backend code still needs to be written manually.

RESEARCH OBJECTIVES

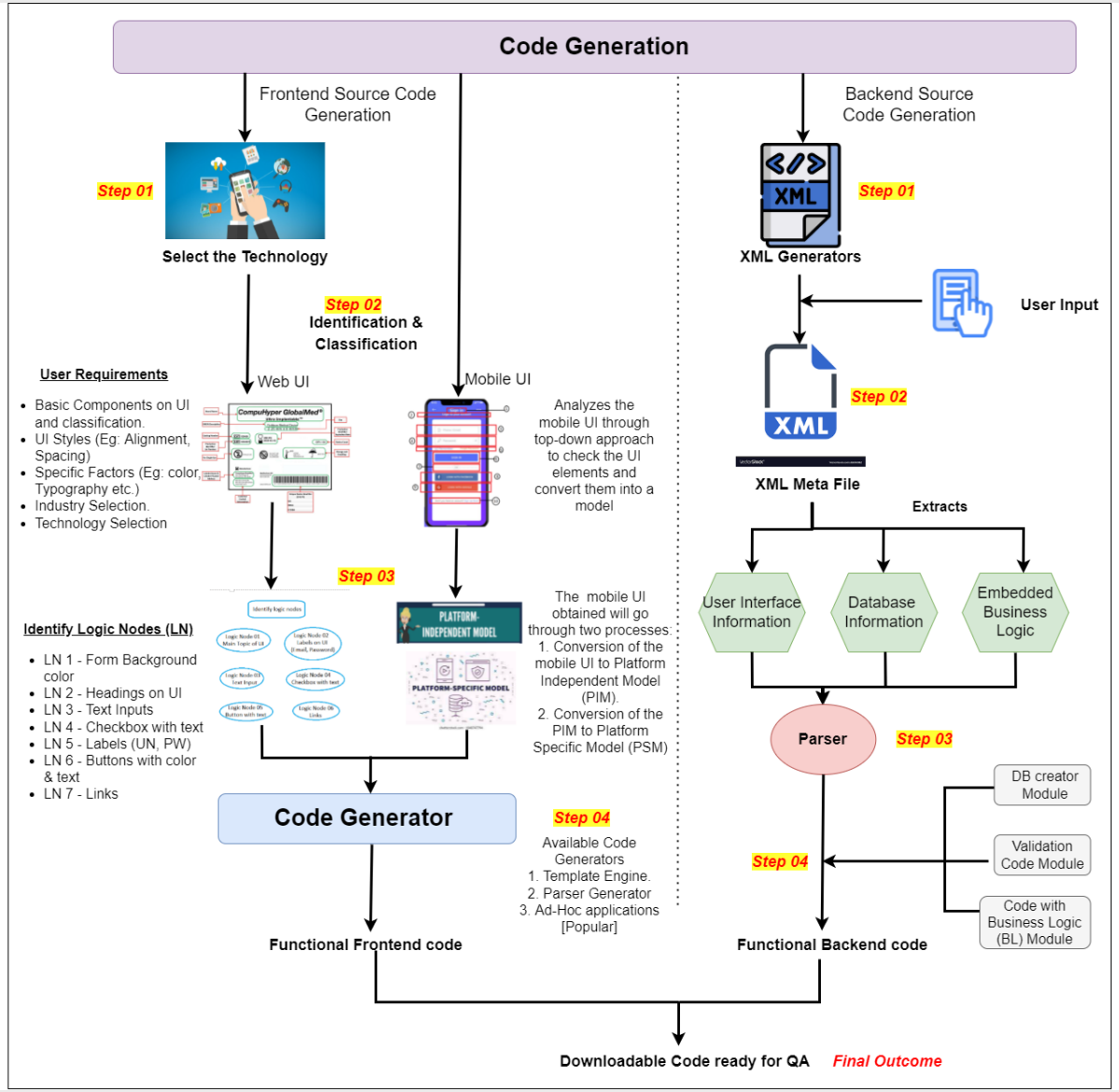
- Generate UI templates according to UI rough sketches given by users.
- Generate web frontend code from UI template using MERN Stack technologies.
- Generate mobile frontend code from UI template using code mapping.
- Develop backend code and DB generator adhering to software engineering principles.



SYSTEM DIAGRAM (PART 01)



SYSTEM DIAGRAM (PART 02)



OUR PLAN TO PROMOTE THE TOOL



- Conducting a live demonstration of code generation process in developer environment.
- Understanding the need and expectations of developers through automated feedback services integrated with the tool.
- Select startup companies and describe them the capabilities and advantages of using UI builder tool for their software development process.
- Use social media and publish an official website for the tool to make it reach out for wide range of people.
- Conduct a workshop on the quick setup of the tool in their PC for 1st and 2nd year IT students.
- Speak around '**Anyone Can Code**' concept to make It stand unique among the other code generation tools.
- Use quick startup guides and YouTube tutorials for better usability and user-friendliness.



IT19150758|EKANAYAKE K.L.W

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

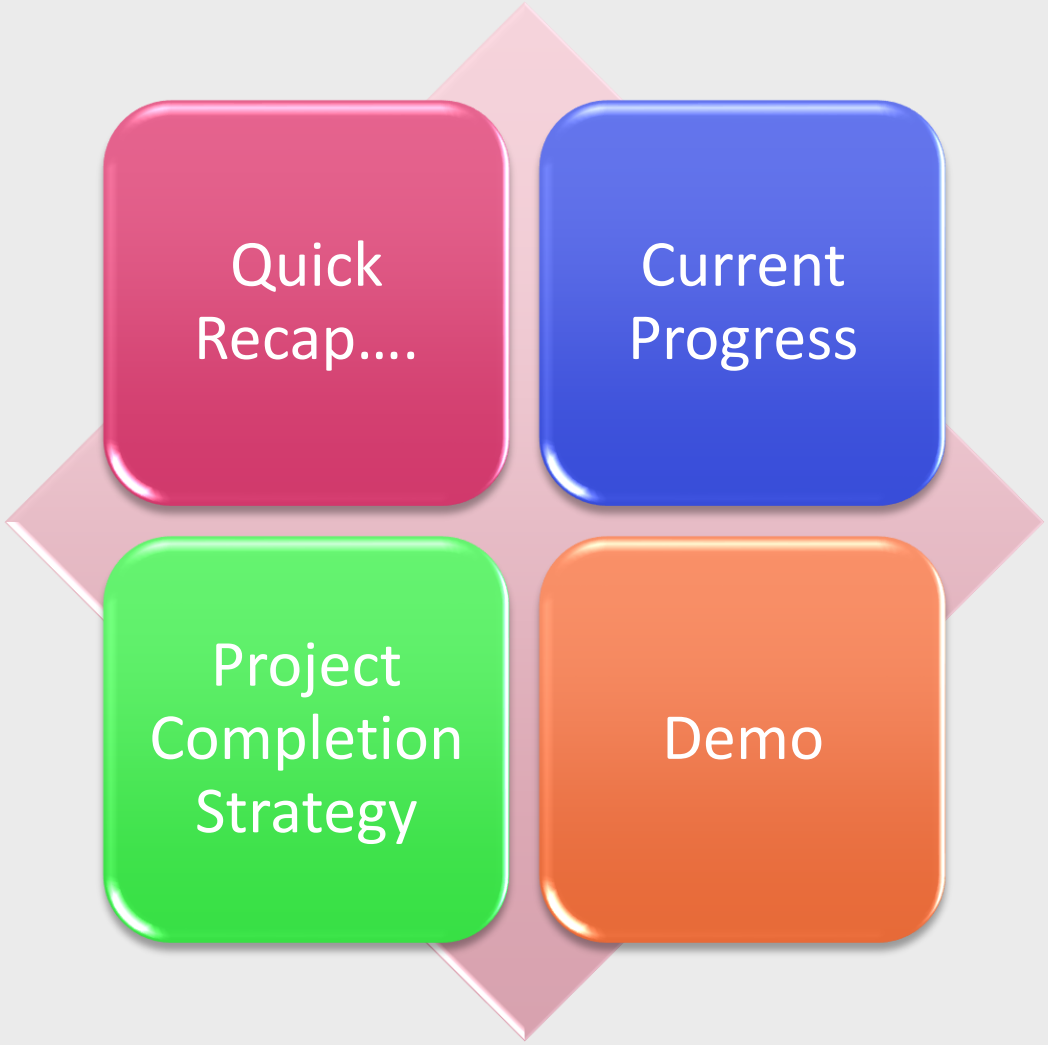
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

GENERATING STANDARD UIs ACCORDING TO ROUGH SKETCHES.



INTRODUCTION



QUICK RECAP.....

Research Question

How to auto-generate UI template from a rough sketch?

1. What are the real user needs?
2. What are the different approaches that can be used?
3. What are the standards and expectations from a UI template generator?

Objectives

Generate a UI template according to a rough sketch.

1. Identify components from the rough sketch.
2. Generate an XML file for the UI.
3. Meet the expectations of the end users.





QUICK RECAP.....

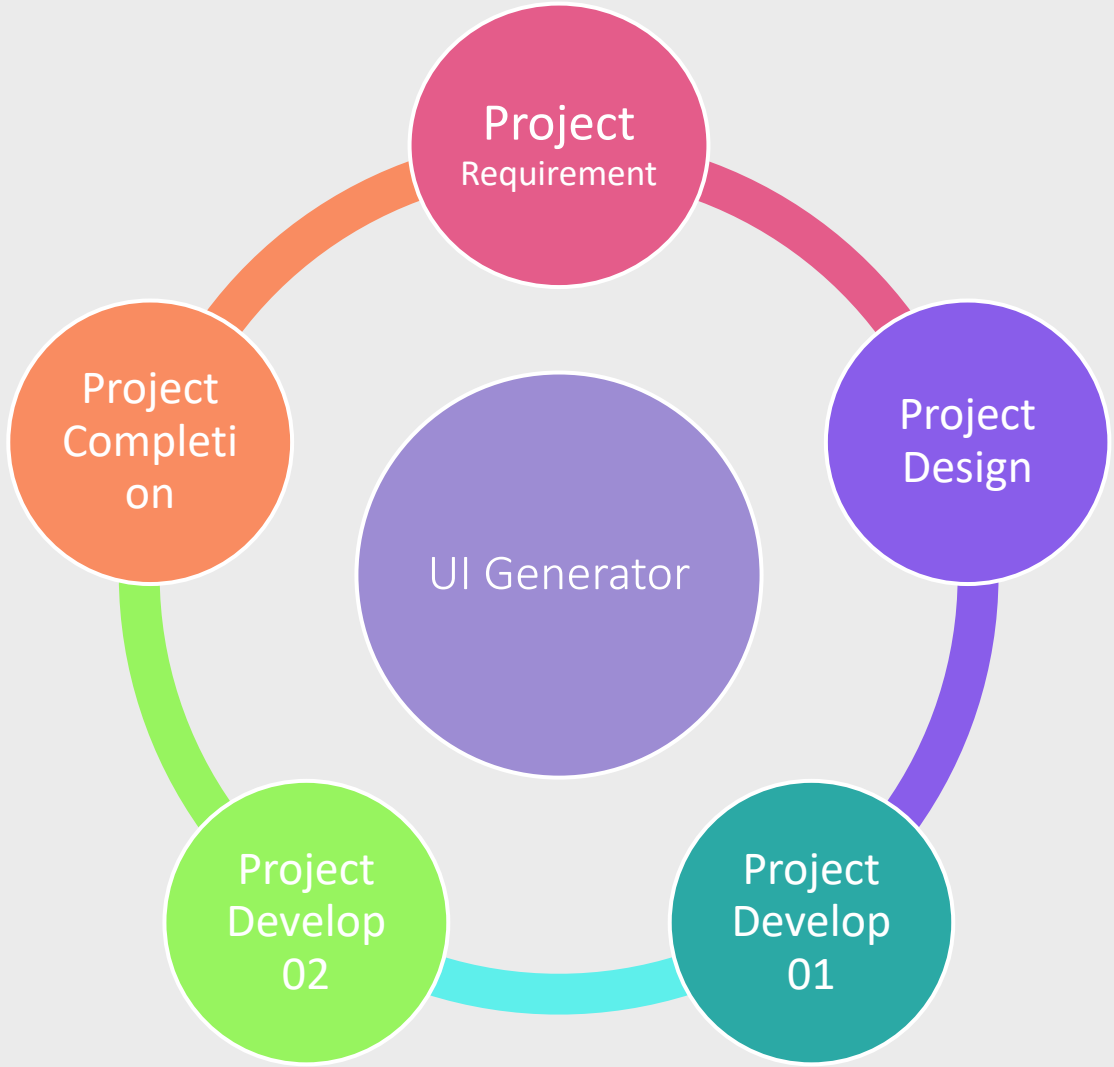
Research Gap

Absence of UI specification documented for future use.

1. Performance & User-friendliness.
2. Flexibility for identifying various components.
3. Identify UI elements through a trained model.
4. Handling multiple UIs.



PROJECT PHASE



CURRENT PROGRESS...

- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Collect a dataset through a survey.
- Create a model CNN(Convolutional Neural Network).
- Train, test and evaluate the model.



REQUIREMENTS - FUNCTIONAL

1. Collect the data.
2. Create a model.
3. Train model.
4. Test **model**.
5. Evaluate **model**.

"Continuous discussion with supervisor helped to sort requirements and finalize"



DESIGN.

Sketching



- Used pen & paper to sketch the UI.
- Identified different components that is essential to implement features.
- Decided on UI navigation.

Wireframing

- Used Mock Flow to draw wireframes for different interfaces.
- Layout and alignment of components in user interface was decided.

Prototyping

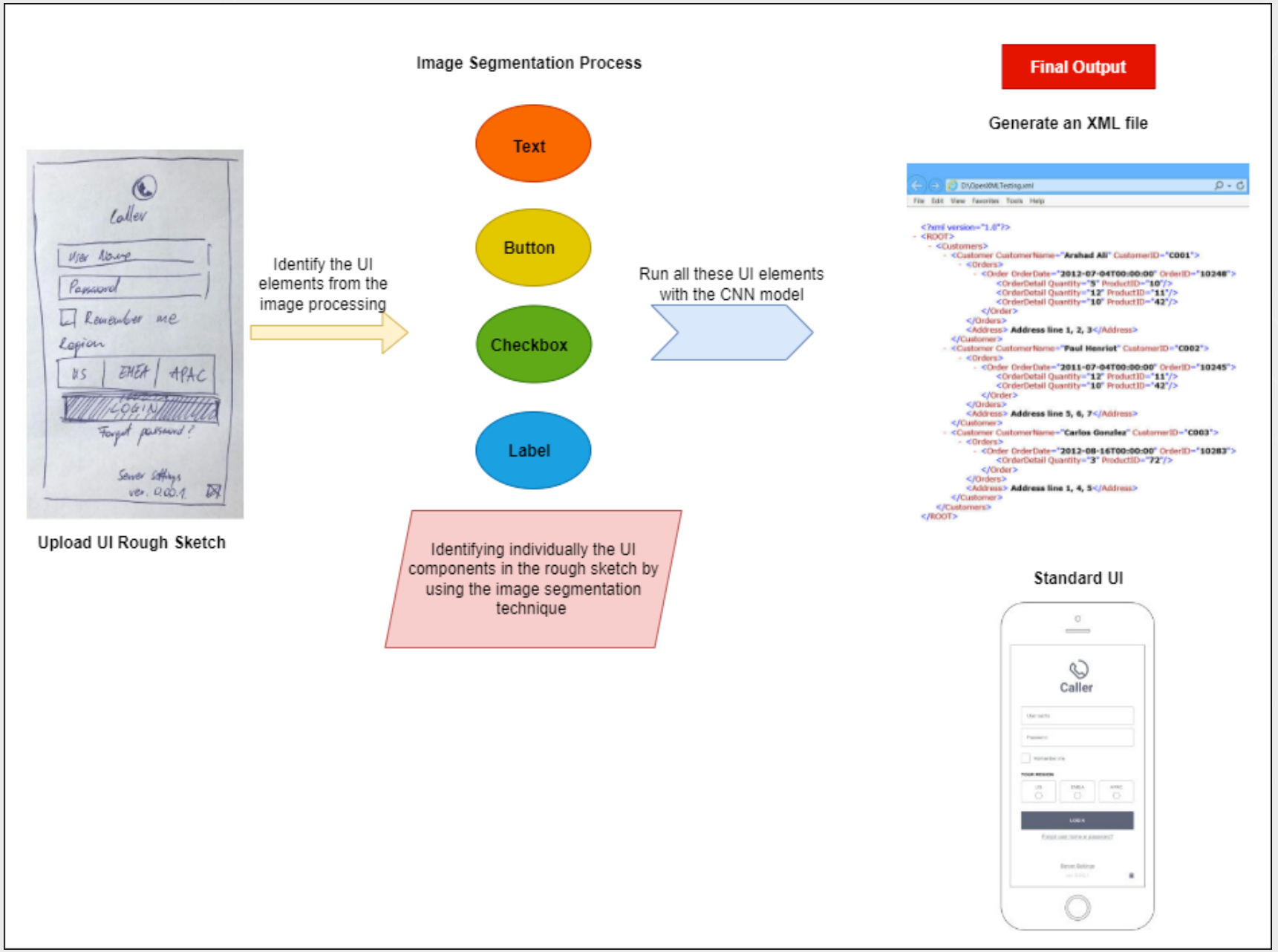
- A prototyping was created using Figma.
- Colors and styles used in user interfaces was decided.
- The prototype served as the best evidence to show how the final output looks like.

IMPLEMENTATION – PHASE 01.



1. Collected the dataset
2. Developed CNN model.
3. Trained the model.
4. Evaluate the model.
5. Developed the frontend UI

PROCESS


Image Segmentation Process
Final Output
Generate an XML file
Text
Button
Checkbox
Label
Identify the UI elements from the image processing
Run all these UI elements with the CNN model
Upload UI Rough Sketch
Identifying individually the UI components in the rough sketch by using the image segmentation technique
Standard UI
Caller

IMPLEMENTATION – PHASE 01 (EVIDENCES)

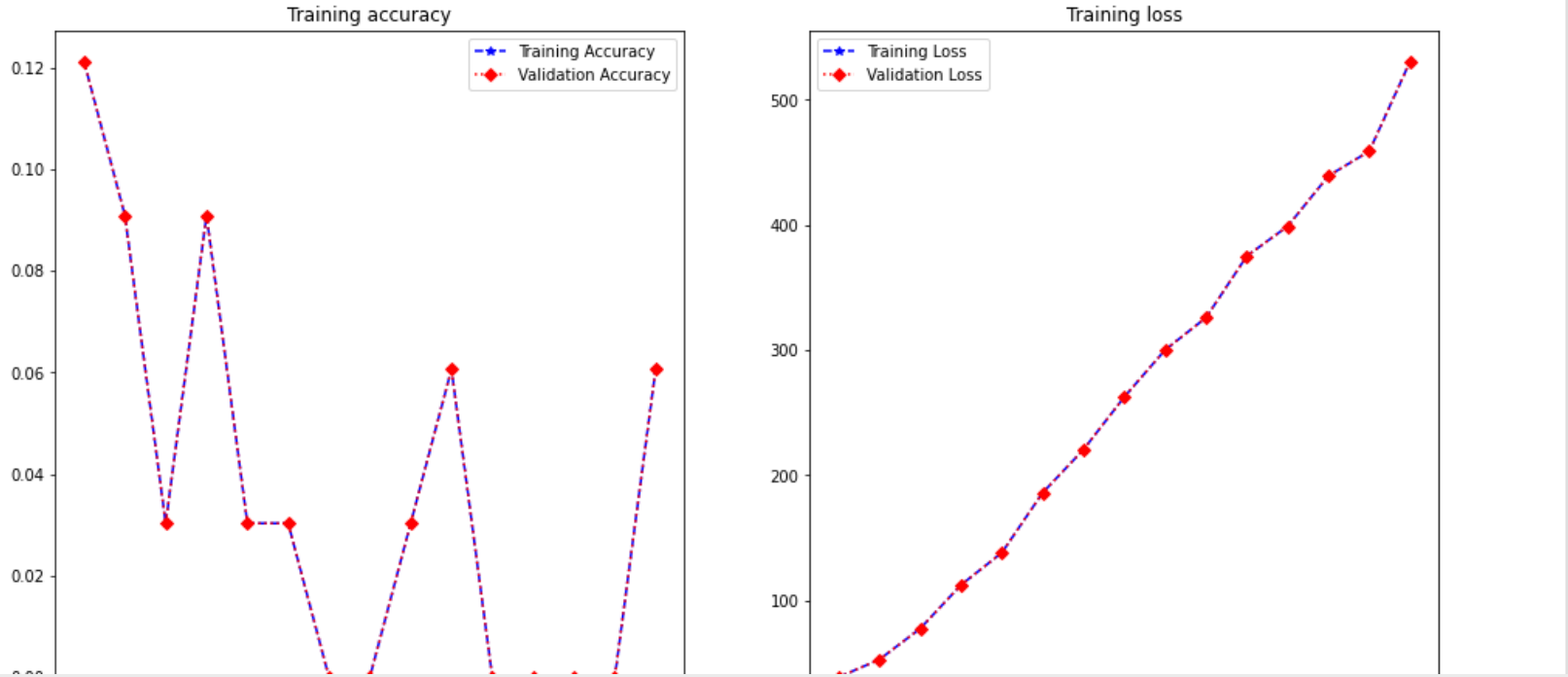
jupyter ComponentClassificationModel_WireframesOnly Last Checkpoint: 2 minutes ago (unsaved changes) Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

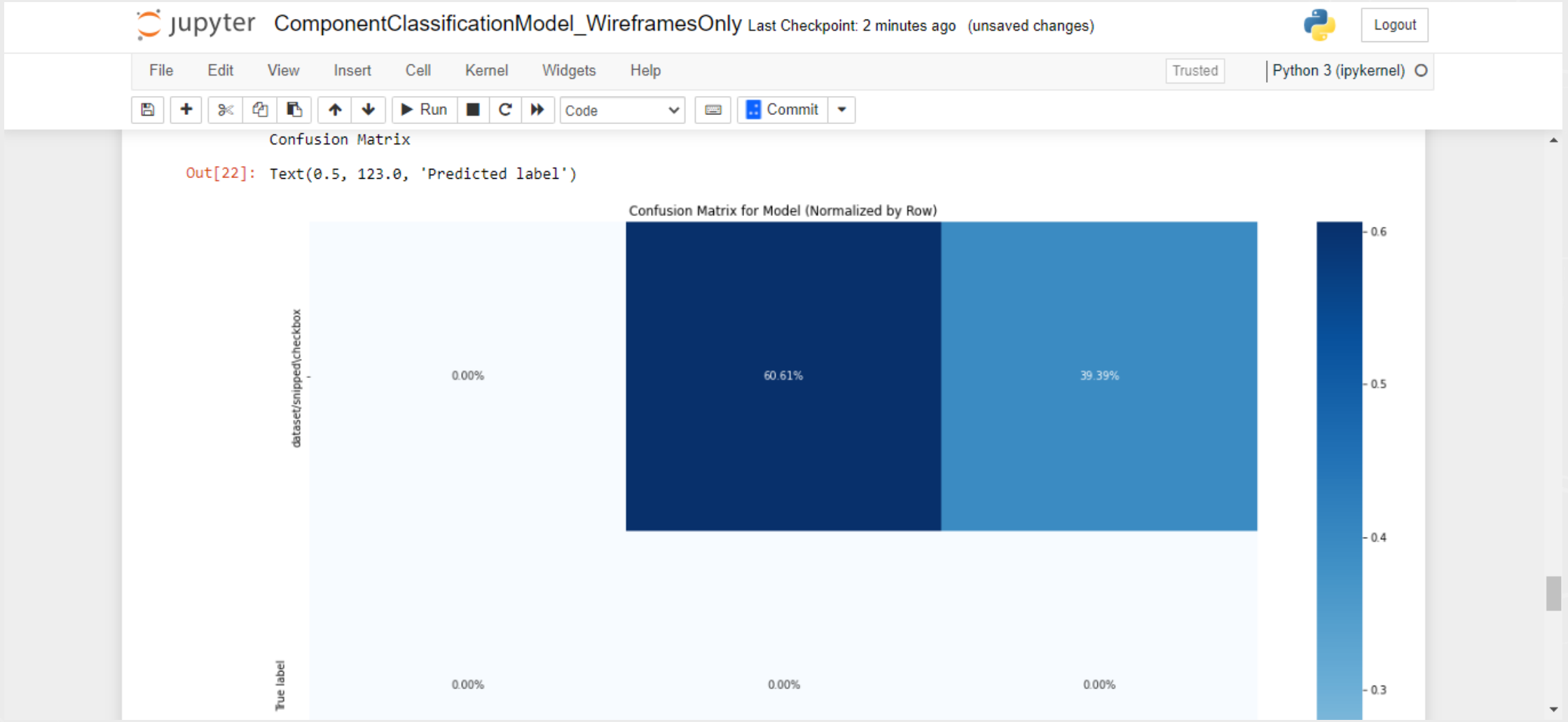
Run Code Commit

```
    plt.show()
```

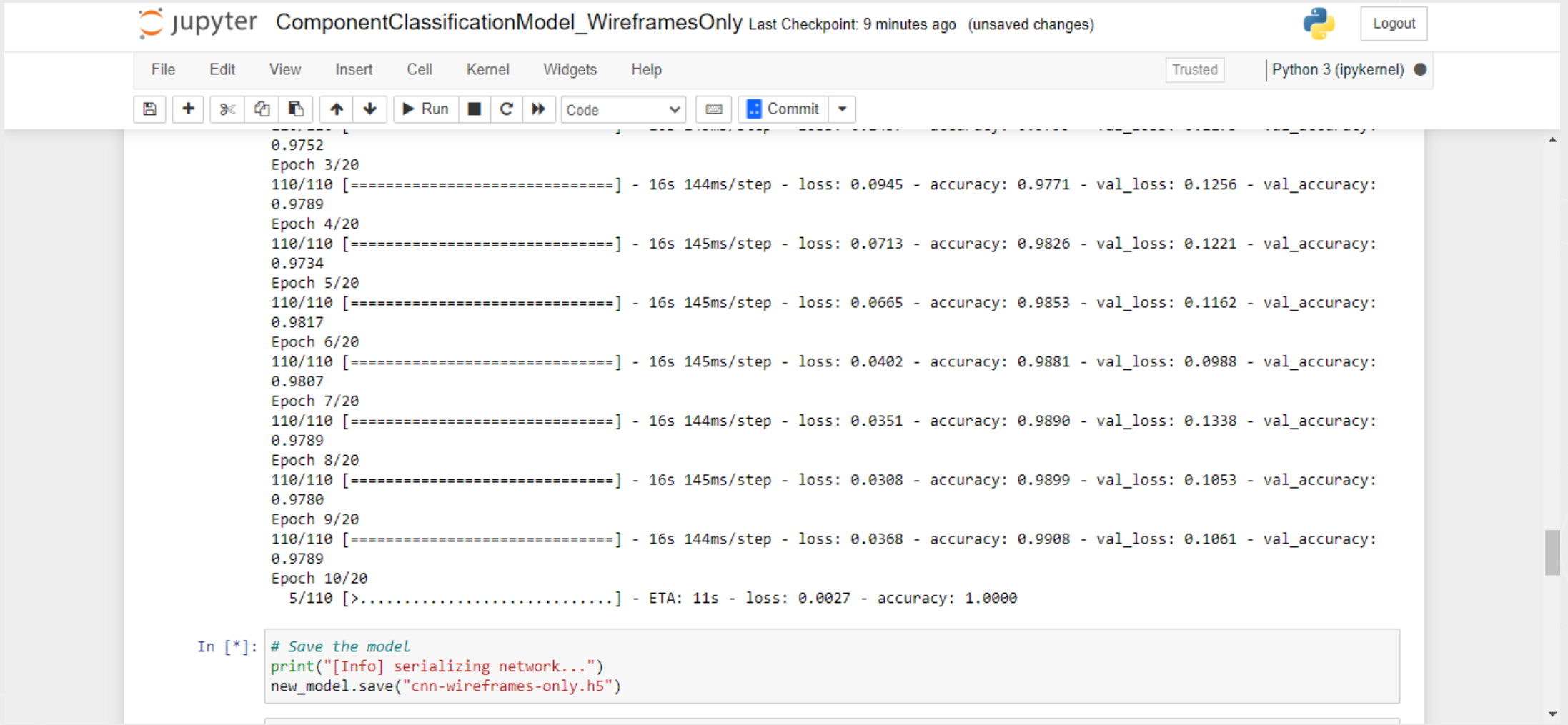
In [18]:
```
plot_training(history)
```



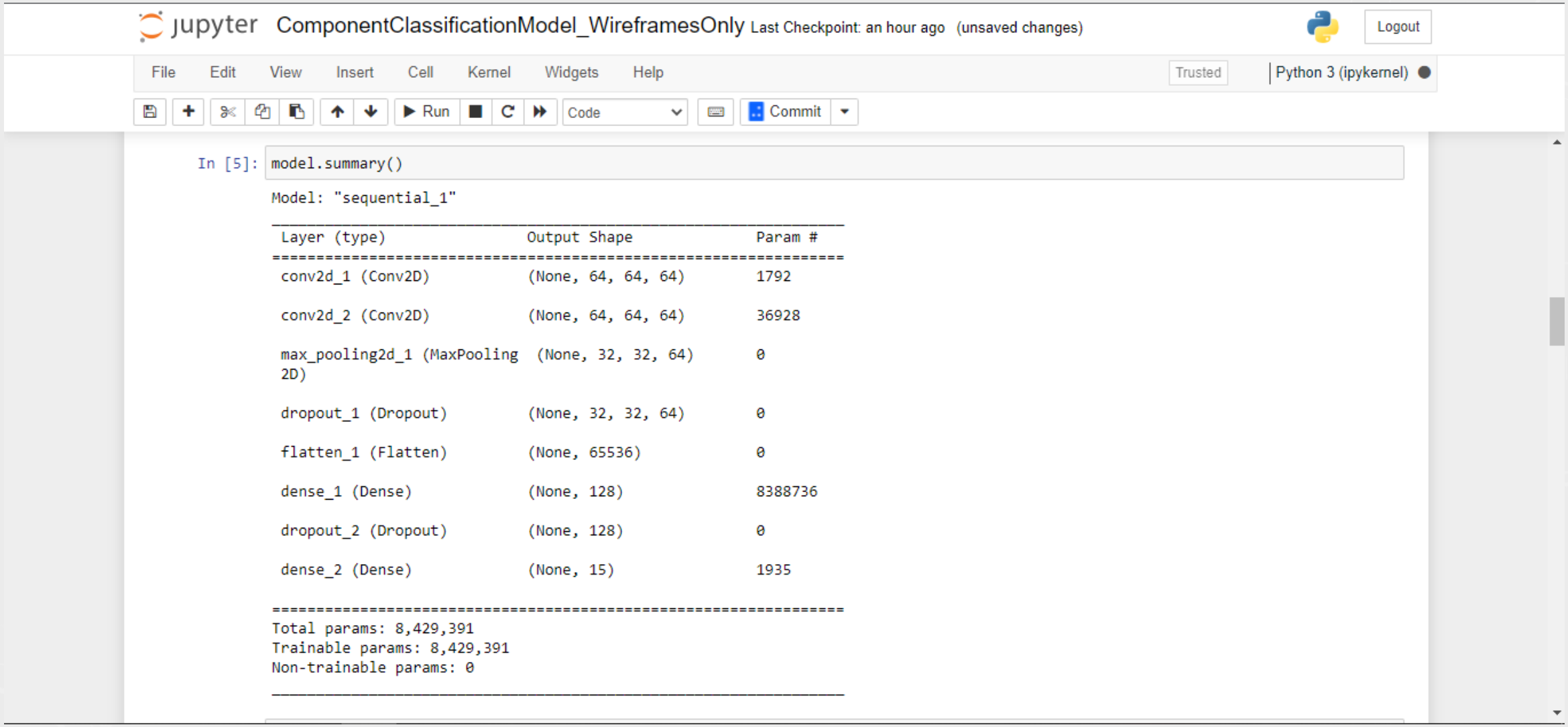
IMPLEMENTATION – PHASE 01 (EVIDENCES)



IMPLEMENTATION – PHASE 01 (EVIDENCES)



IMPLEMENTATION – PHASE 01 (EVIDENCES)



IMPLEMENTATION – PHASE 01 (EVIDENCES)

jupyter ComponentClassificationModel_WireframesOnly Last Checkpoint: an hour ago (unsaved changes) Logout

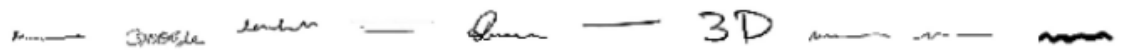
File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)

Run Code Commit

In [13]:
```
# Plot some images form the augmented results

x_batch, y_batch = next(training_set)

plt.figure(figsize=(12, 9))
for k, (img, lbl) in enumerate(zip(x_batch, y_batch)):
    # img = (img - np.min(img))/(np.max(img) - np.min(img))
    plt.subplot(5, 10, k+1)
    # im = Image.open(img).convert('RGB')
    plt.imshow(np.asarray(img)[:, :, ])
    plt.axis('off')
```

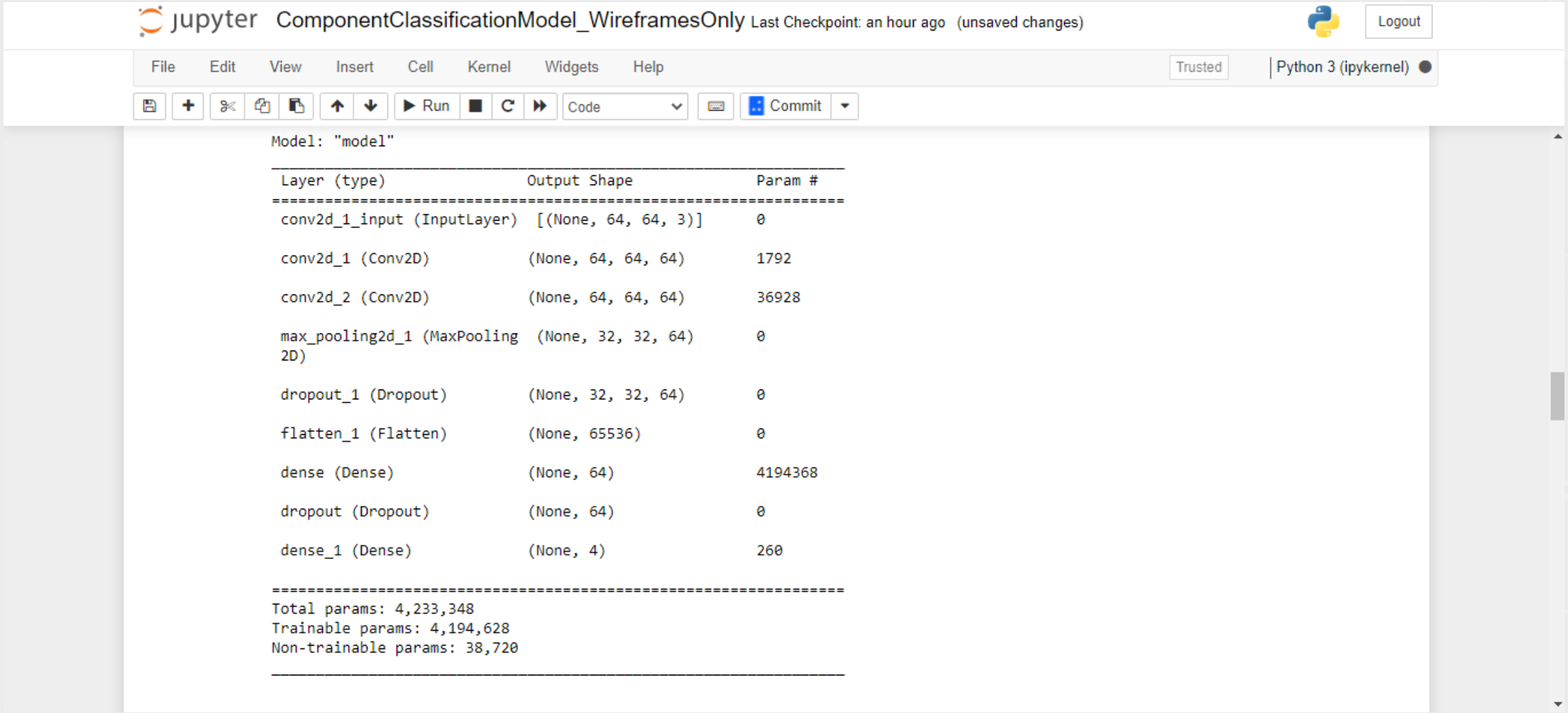


Model Training

In [14]:
```
new_model.compile(
    loss="categorical_crossentropy",
    optimizer = Adam(lr=0.001),
    metrics=["accuracy"]
)
```

```
C:\Users\Keshawa Ekanayake\AppData\Local\Programs\Python\Python38\lib\site-packages\keras\optimizers\optimizer_v2\adam.py:110:
```

IMPLEMENTATION – PHASE 01 (EVIDENCES)




```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_1_input (InputLayer)  [(None, 64, 64, 3)]      0

 conv2d_1 (Conv2D)           (None, 64, 64, 64)        1792

 conv2d_2 (Conv2D)           (None, 64, 64, 64)        36928

 max_pooling2d_1 (MaxPooling  (None, 32, 32, 64)       0
 2D)

 dropout_1 (Dropout)         (None, 32, 32, 64)        0

 flatten_1 (Flatten)         (None, 65536)             0

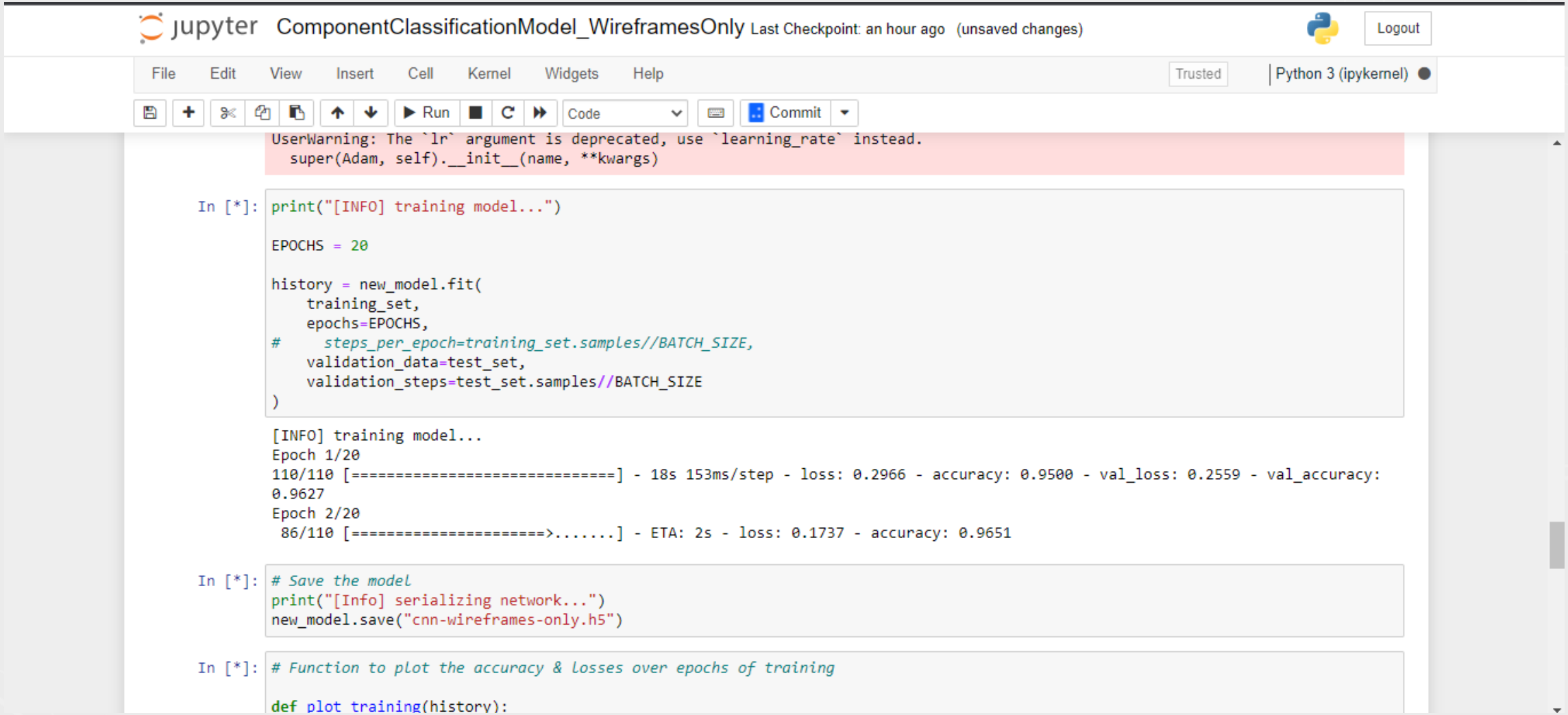
 dense (Dense)               (None, 64)                4194368

 dropout (Dropout)           (None, 64)                0

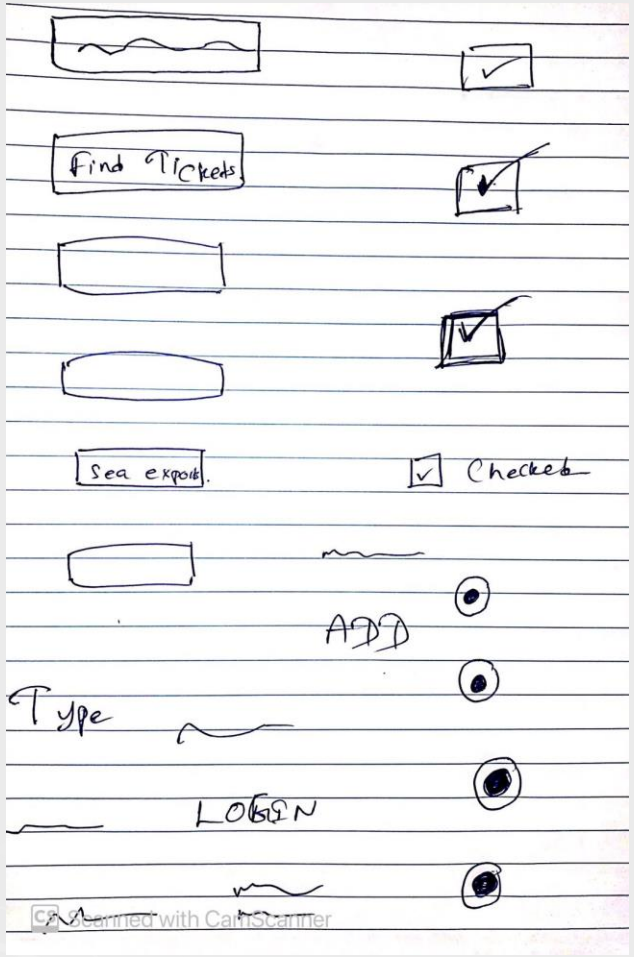
 dense_1 (Dense)             (None, 4)                 260

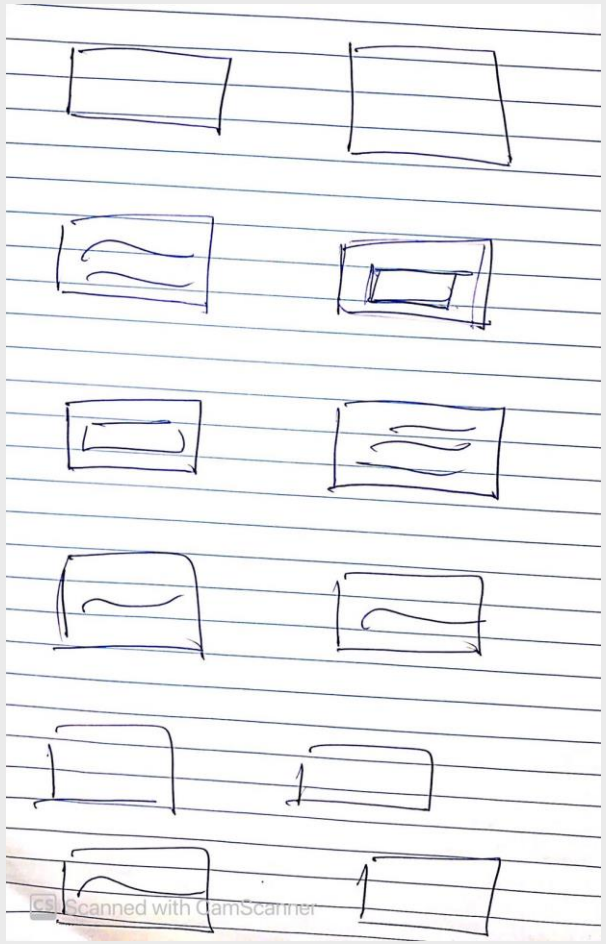
=================================================================
Total params: 4,233,348
Trainable params: 4,194,628
Non-trainable params: 38,720
_________________________________________________________________
```

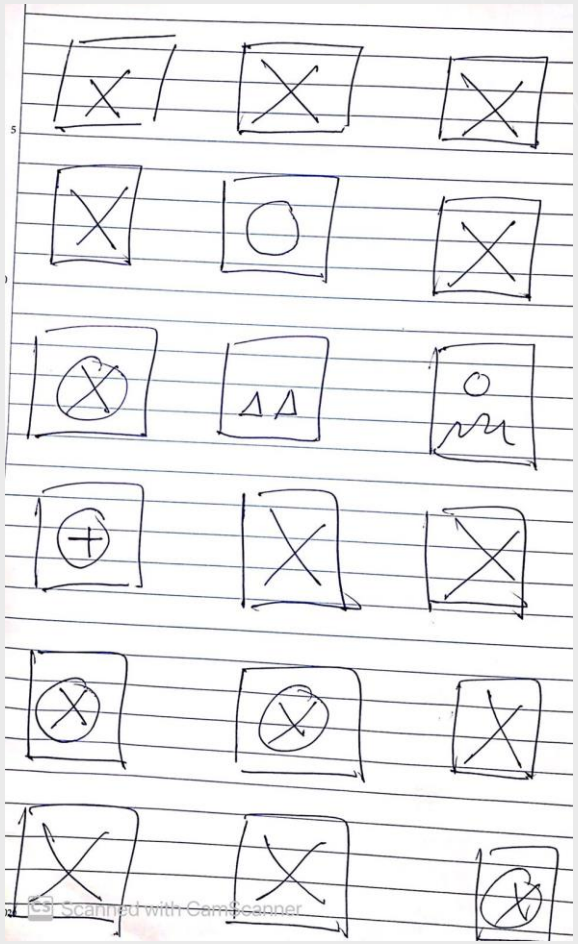
IMPLEMENTATION – PHASE 01 (EVIDENCES)



IMPLEMENTATION – PHASE 01 (EVIDENCES) COLLECTED DATA





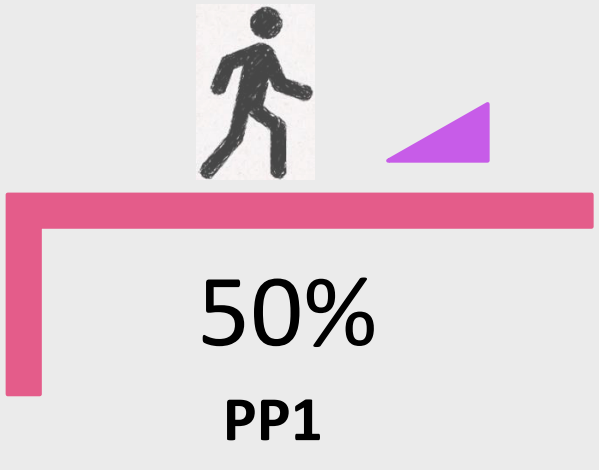


IMPLEMENTATION – PHASE 02.



- Image segmentation
- Complete the frontend.
- Integration
- Try to add the new features addressed by Nuwan sir..

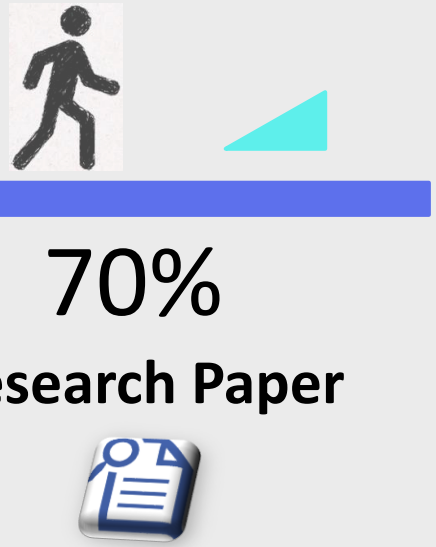
PROJECT COMPLETION STRATEGY



50%

PP1

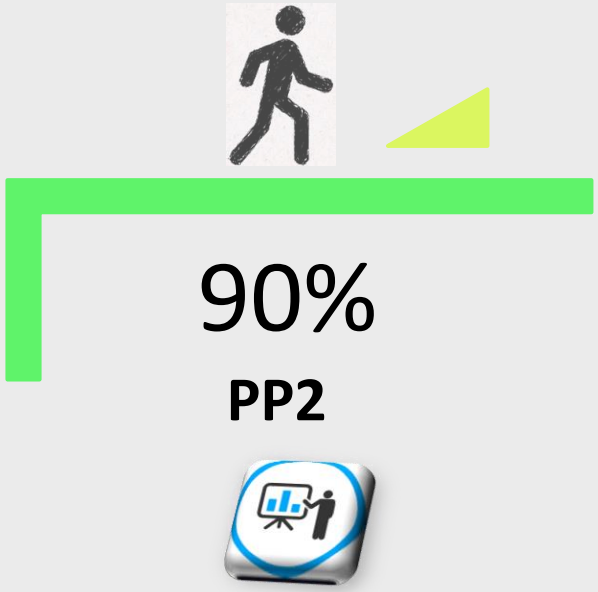
- Requirements & Design.
- Project Implementation – PHASE 01



70%

Research Paper

- Identify rough sketches
- Handle validations and user flows
- Unit Testing



90%

PP2

- Completion of Project Implementation – PHASE 02
- Integrate with other components
- Integration Testing



100%

Final Presentation

- System Testing
- Project Completion



IT19158228|SAKTHIVEL.S

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

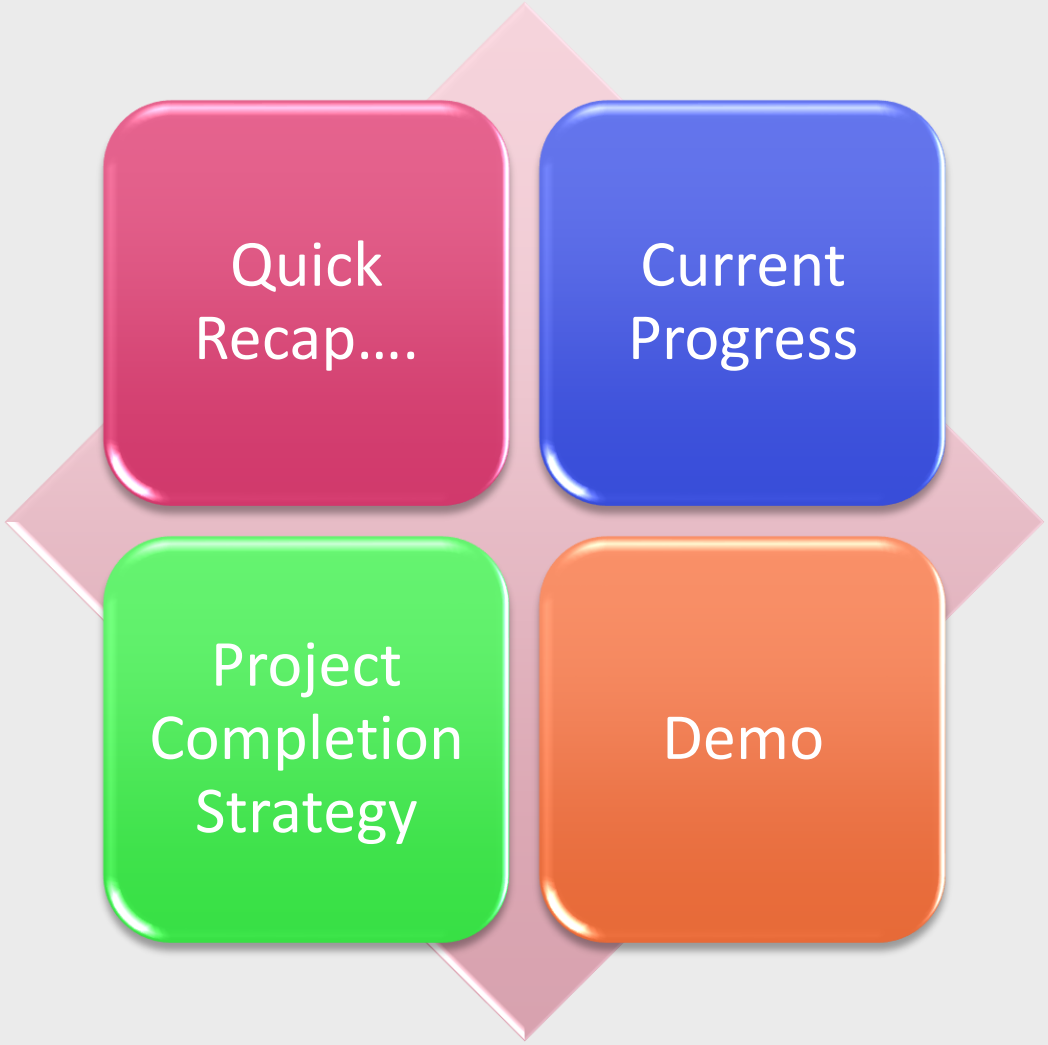
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

BACKEND CODE AND DATABASE GENERATOR



INTRODUCTION



QUICK RECAP.....

Research Question

How to auto-generate backend code with database and deliver it to end user?

1. What are the real user needs?
2. What are the different approaches that can be used?
3. What are the standards and expectations from a auto-generated code?



Objectives

Generate a fully functional backend code with database and configuration files.

1. Easily understandable code generation process and Continuous Integration (CI) pipeline.
2. Deliver code with good quality and best practices.
3. Code refactoring and customization of generated code by end users.
4. Code base should be clean and easy to add features on top.



QUICK RECAP.....

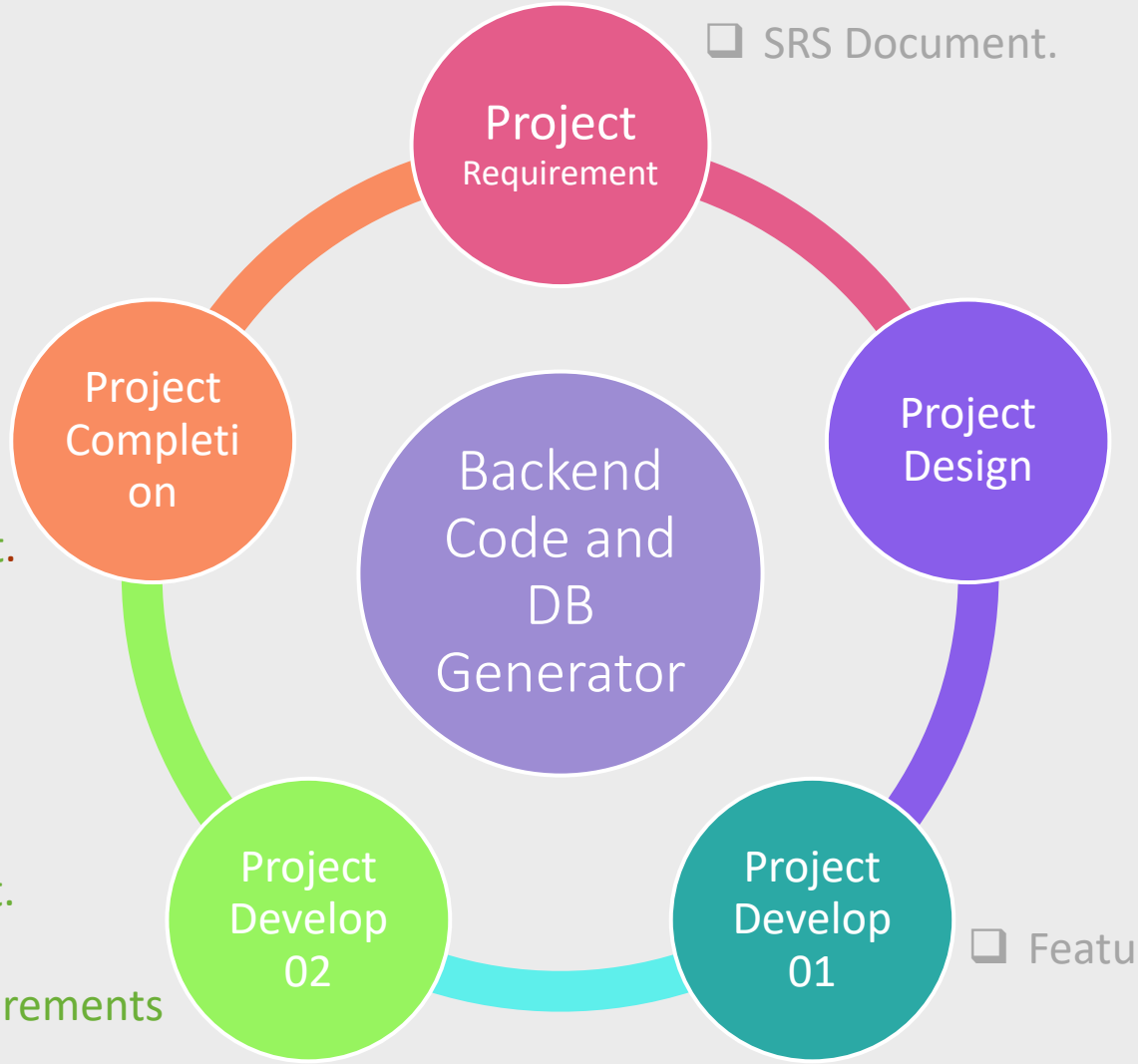
Research Gap

Absence of automation in the direction of backend code generation

1. Available ACG tools mainly target on frontend code.
2. Frontend code along with backend connectivity is the real expectation.
3. Flexible and easily manageable Database.
4. Data Integrity and Validations



PROJECT PHASE



- SRS Document.

- UI Sketch
- Wireframing
- Prototyping

- Feature Development.

- Feature Development.
- Navigation.
- Non-Functional Requirements

- Code Refactoring.
- Commenting.
- Code base cleanup.
- Bug Fixes.
- Completed Component.

CURRENT PROGRESS...

- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Write the original XML File and Refined XML File. (UI Descriptor)
- Initiated code generation process with UI navigation.
- Completed Database Connection and Table creation process.
- Accomplished CRUD Code generation & automatic comment generation.



REQUIREMENTS - FUNCTIONAL

1. Connect to Database from UI.
2. Create table in database without writing SQL Query.
3. Generate CRUD Code
4. Generate REST API Code
5. View and Save code to local machine.

"Continuous discussion with supervisor helped to sort requirements and finalize"



DESIGN.

Sketching



- Used pen & paper to sketch the UI.
- Identified different components that is essential to implement features.
- Decided on UI navigation.

Wireframing

MockFlow

- Used Mock Flow to draw wireframes for different interfaces.
- Layout and alignment of components in user interface was decided.

Prototyping

Figma

- A prototyping was created using Figma.
- Colors and styles used in user interfaces was decided.
- The prototype served as the best evidence to show how the final output looks like.

IMPLEMENTATION – PHASE 01.

- Allow users to connect to database, test connection and establish a session.
- Create tables in database by reading the refined XML.
- Store table modifications for future use.
- Generate Code and Comments for basic CRUD operations.
- Initiated the REST API Code generation.
- View the generated
- Get the code to own PC.





1. How did you manage to get the desired outcome?
2. What is the process you follow?
3. What is a refined XML and its purpose?

APPROACH.

- Tried different approaches on finding how code generated.
- Quote from Nuwan sir

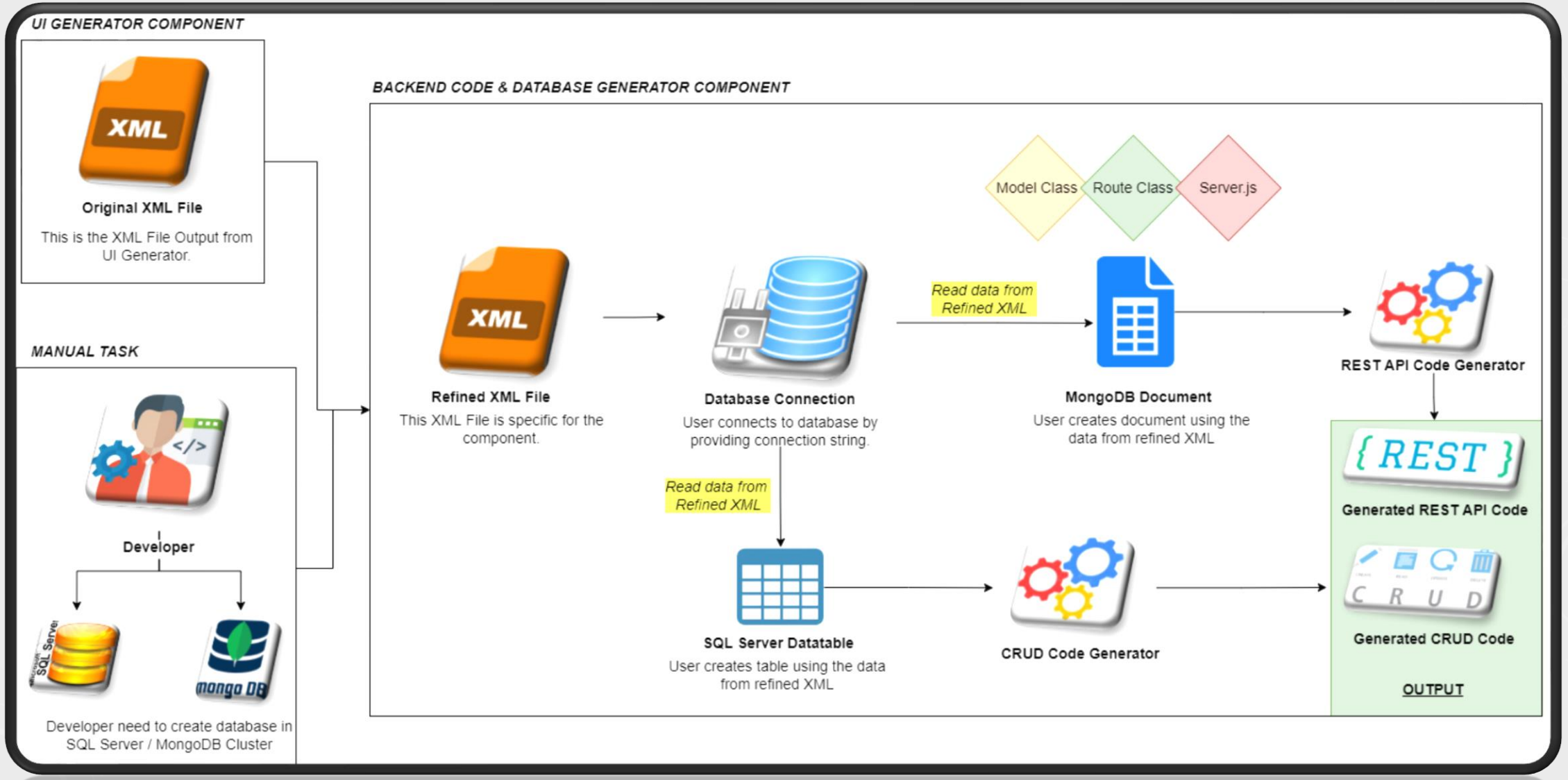
  "Code generation is not a magic, it's the process of applying your SE skills"

  – This was the initial motivation
- **Approach : Generate Code using wizards (Windows)**
- Advantage:

  1. From the survey, end users need to understand the code generation process.
  2. As mentioned before, code is not a magic, it can even fail in some case.
  3. How well the user focuses on the process.



PROCESS


UI GENERATOR COMPONENT
XML
Original XML File
This is the XML File Output from UI Generator.
MANUAL TASK
Developer
Developer need to create database in SQL Server / MongoDB Cluster
BACKEND CODE & DATABASE GENERATOR COMPONENT
Model Class
Route Class
Server.js
XML
Refined XML File
This XML File is specific for the component.
Database Connection
User connects to database by providing connection string.
Read data from Refined XML
MongoDB Document
User creates document using the data from refined XML
REST API Code Generator
{ REST }
Generated REST API Code
Read data from Refined XML
SQL Server Datatable
User creates table using the data from refined XML
CRUD Code Generator
C R U D
Generated CRUD Code
OUTPUT

SAMPLE REFINED XML FILE

```xml
<?xml version="1.0" encoding="utf-8"?>
<Table>
    <Name>Customer</Name>
    <Columns>
      <Column>
        <Name>CustomerID</Name>
        <ColumnName>custId</ColumnName>
        <DataType>nvarchar</DataType>
        <Length>4</Length>
        <PrimaryKey>true</PrimaryKey>
      </Column>
      <Column>
        <Name>Customer Name</Name>
        <ColumnName>custName</ColumnName>
        <DataType>nvarchar</DataType>
        <Length>50</Length>
      </Column>
      <Column>
        <Name>Customer Address</Name>
        <ColumnName>custAddress</ColumnName>
        <DataType>nvarchar</DataType>
        <Length>150</Length>
      </Column>
      </Column>
      <Column>
        <Name>Customer Telephone</Name>
        <ColumnName>cutsTel</ColumnName>
        <DataType>nvarchar</DataType>
        <Length>10</Length>
      </Column>
      <Column>
        <Name>Customer DOB</Name>
        <ColumnName>dateOfBirth</ColumnName>
        <DataType>nvarchar</DataType>
        <Length>25</Length>
      </Column>
    </Columns>
</Table>
```

BENEFITS OF REFINED XML

- Modification on original XML is not efficient and may result in in-accurate read and writes.
- Refined XML is specific for purpose – DESCRIBE THE TABLE STRUCTURE.
- Table structure is subjected to change which user needs to track. Refined XML is the right way.

  (Really need to thank Nuwan sir for his suggestion on this, because he identified this requirement after development was complete)
- And changes or modification on refined XML do not affect other components in the tool.

Finally, as all developers usually think, software applications can change time to time, so static code is not possible!!!!!!

SAMPLE GENERATED CODE - VIEW IN VS CODE

```
/*
**********************************************************************
*NOTE          :   This is an auto-generated Code
*@Developer    :   (Add Developer Name Here)
*@Purpose      :   Add Customer to database table.
*@Created Date :   5/8/2022 6:48:57 PM
**********************************************************************
*/
public void addCustomer(String custId, String custName, String custAddress, String cutsTel, String dateOfBirth) {
    try{
        SqlConnection con = new SqlConnection(@Data Source=DESKTOP-I53H1O7;Initial Catalog='Test Database';Integrated Security=True);
        SqlCommand cmd;

        String sql = "INSERT INTO Customer VALUES(@custId,@custName,@custAddress,@cutsTel,@dateOfBirth);"
        con.Open();
        cmd = new SqlCommand(sql, con);

        cmd.Parameters.AddWithValue("@custId", custId);
        cmd.Parameters.AddWithValue("@custName", custName);
        cmd.Parameters.AddWithValue("@custAddress", custAddress);
        cmd.Parameters.AddWithValue("@cutsTel", cutsTel);
        cmd.Parameters.AddWithValue("@dateOfBirth", dateOfBirth);

        cmd.ExecuteNonQuery();

        Console.Write('Date Added Successfully!');

    } catch(Exception ex) {
        Console.Write("Exception Occurred!");
    }

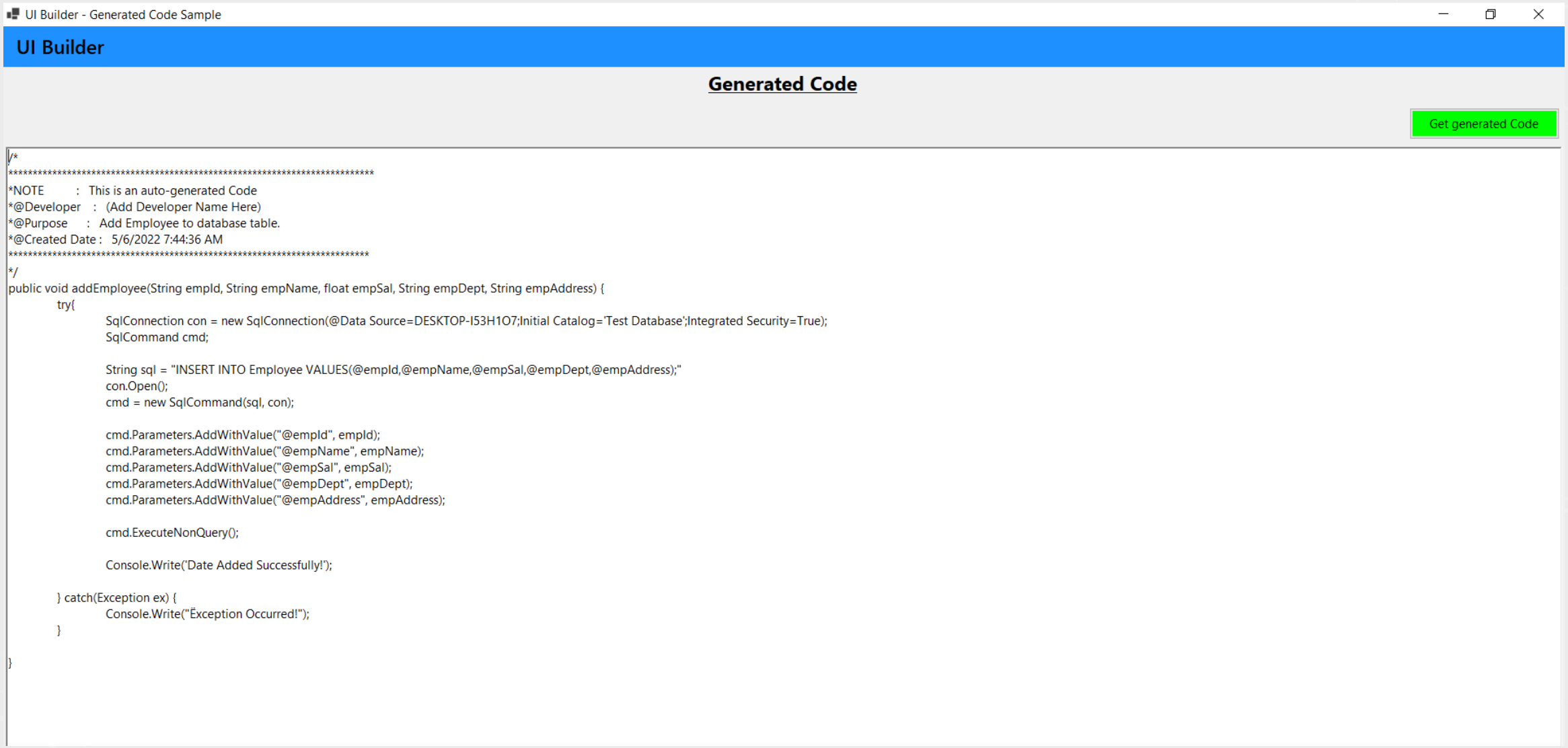
}
```



Observe!

1. **Comments.**
2. **Method Signature.**
3. **Syntax.**
4. **Arrange of Code.**
5. **Error Handling.**

SAMPLE GENERATED CODE – IN APPLICATION




```
/*
**********************************************************************************
*NOTE        :  This is an auto-generated Code
*@Developer   :  (Add Developer Name Here)
*@Purpose     :  Add Employee to database table.
*@Created Date :  5/6/2022 7:44:36 AM
**********************************************************************************
*/
public void addEmployee(String empId, String empName, float empSal, String empDept, String empAddress) {
        try{
                SqlConnection con = new SqlConnection(@Data Source=DESKTOP-I53H1O7;Initial Catalog='Test Database';Integrated Security=True);
                SqlCommand cmd;

                String sql = "INSERT INTO Employee VALUES(@empId,@empName,@empSal,@empDept,@empAddress);"
                con.Open();
                cmd = new SqlCommand(sql, con);

                cmd.Parameters.AddWithValue("@empId", empId);
                cmd.Parameters.AddWithValue("@empName", empName);
                cmd.Parameters.AddWithValue("@empSal", empSal);
                cmd.Parameters.AddWithValue("@empDept", empDept);
                cmd.Parameters.AddWithValue("@empAddress", empAddress);

                cmd.ExecuteNonQuery();

                Console.Write('Date Added Successfully!');

        } catch(Exception ex) {
                Console.Write("Exception Occurred!");
        }

}
```

IMPLEMENTATION – PHASE 02.

- Auto-generate SQL Server and MongoDB Connection URL.
- Completion of REST API Code generation process.
- Code refactoring.
- Commenting and code base cleanup.
- Test with real user and control environment.
- **Integration**.



PROJECT COMPLETION STRATEGY



50%

PP1

- Requirements & Design.
- Project Implementation – PHASE 01



70%

Research Paper



- Complete REST API Code Generation
- Handle validations and user flows
- Unit Testing



90%

PP2



- Completion of Project Implementation – PHASE 02
- Integrate with other components
- Integration Testing



100%

Final Presentation



- System Testing
- Project Completion

HOPE YOU GOT AN IDEA, BUT STILL DEMO HELPS YOU SEE THE REAL PROCESS.

TRY TO KEEP IN MIND FOR THE DEMO…….

- ❑ Component can **generate SQL Server or MongoDB Code.**
- ❑ Component has its **own Refined XML.**
- ❑ Component can **connect to DB automatically & create tables automatically.**
- ❑ Component is capable to **handle changes in table structure**.
- ❑ Component can **generate CRUD or REST API Code.**

Many more…. But I don't want to stress you! More presentations left ☺



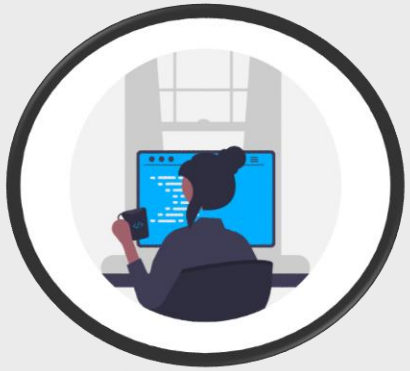
IT19144986|H.M. KASUNI NAVODYA

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

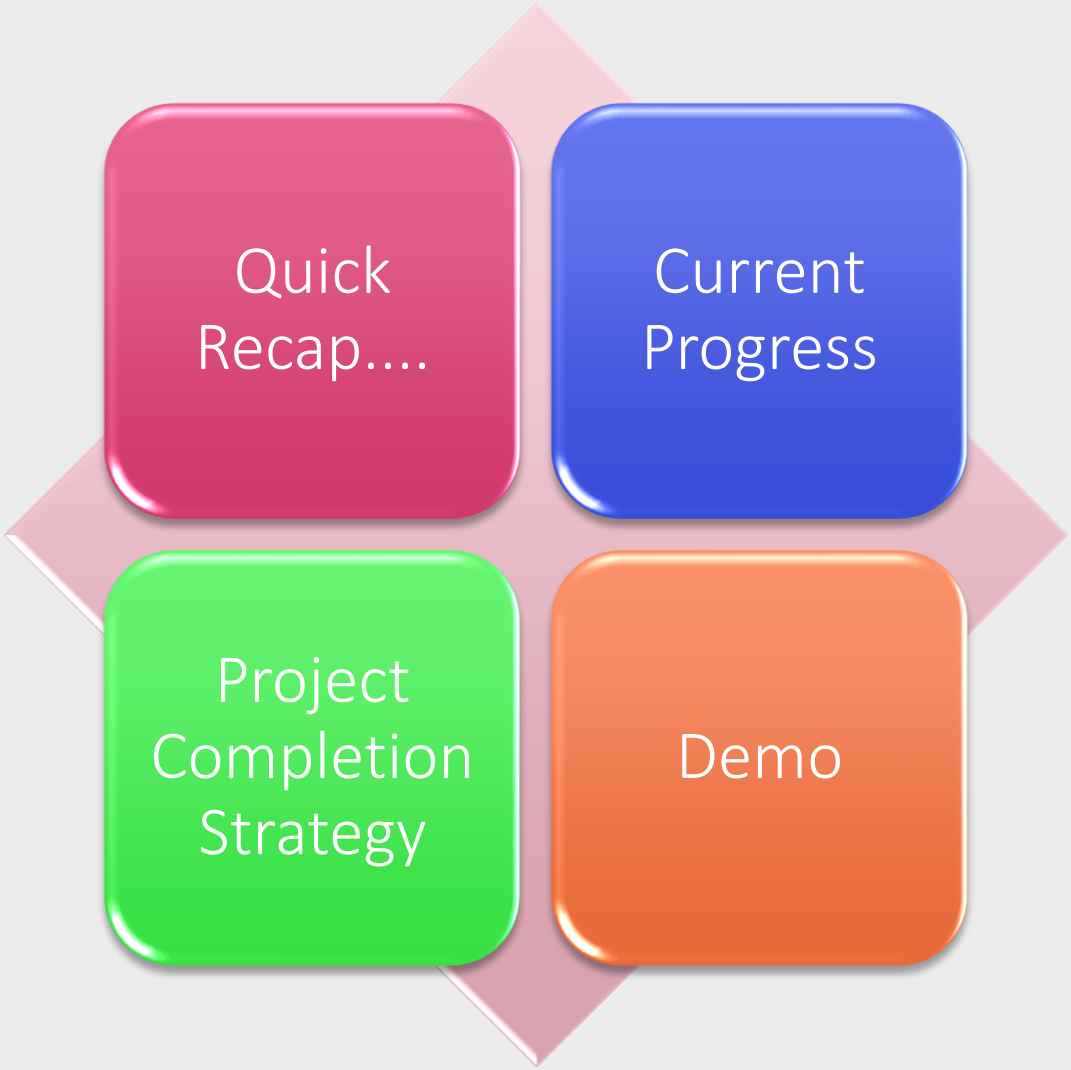
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

FRONT-END CODE GENERATION FOR WEB APPLICATIONS.



INTRODUCTION



QUICK RECAP.....

Research Question

How to generate efficient and unique automated web frontend code?

1. What are the best frontend technologies that user prefers?
2. What are the challenges faced by users in web frontend development?
3. What are the standards and other expectations from an auto-generated web frontend code?



Objectives

Generate a simple and accurate automated frontend code for web applications.

1. Easily understandable and simple frontend code generation process.
2. Deliver code with user-friendliness and best practices.
3. Allow customization of the generated code for end users.



QUICK RECAP.....

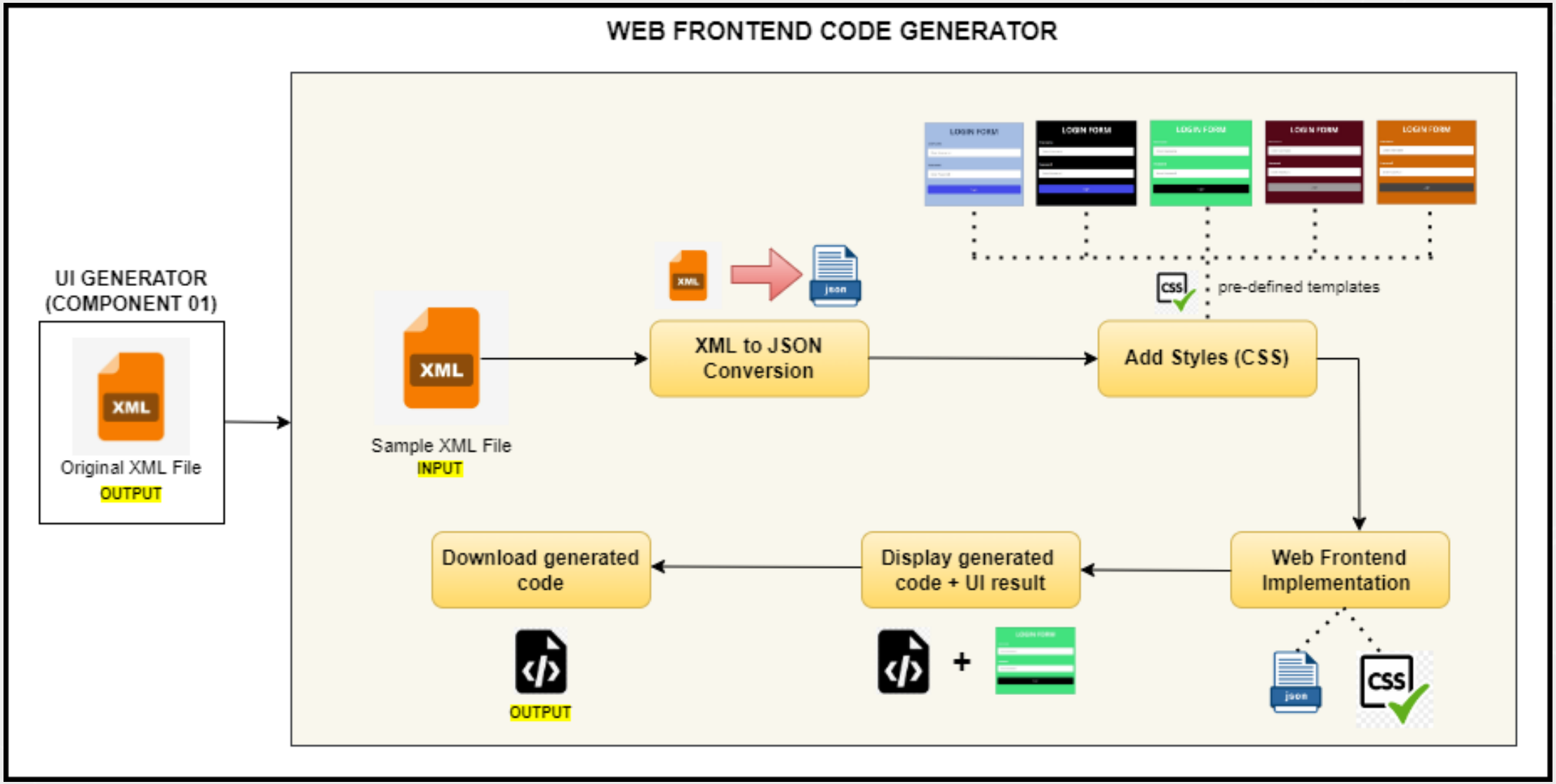
Research Gap

Absence of flexible & scalable web frontend code generation process.

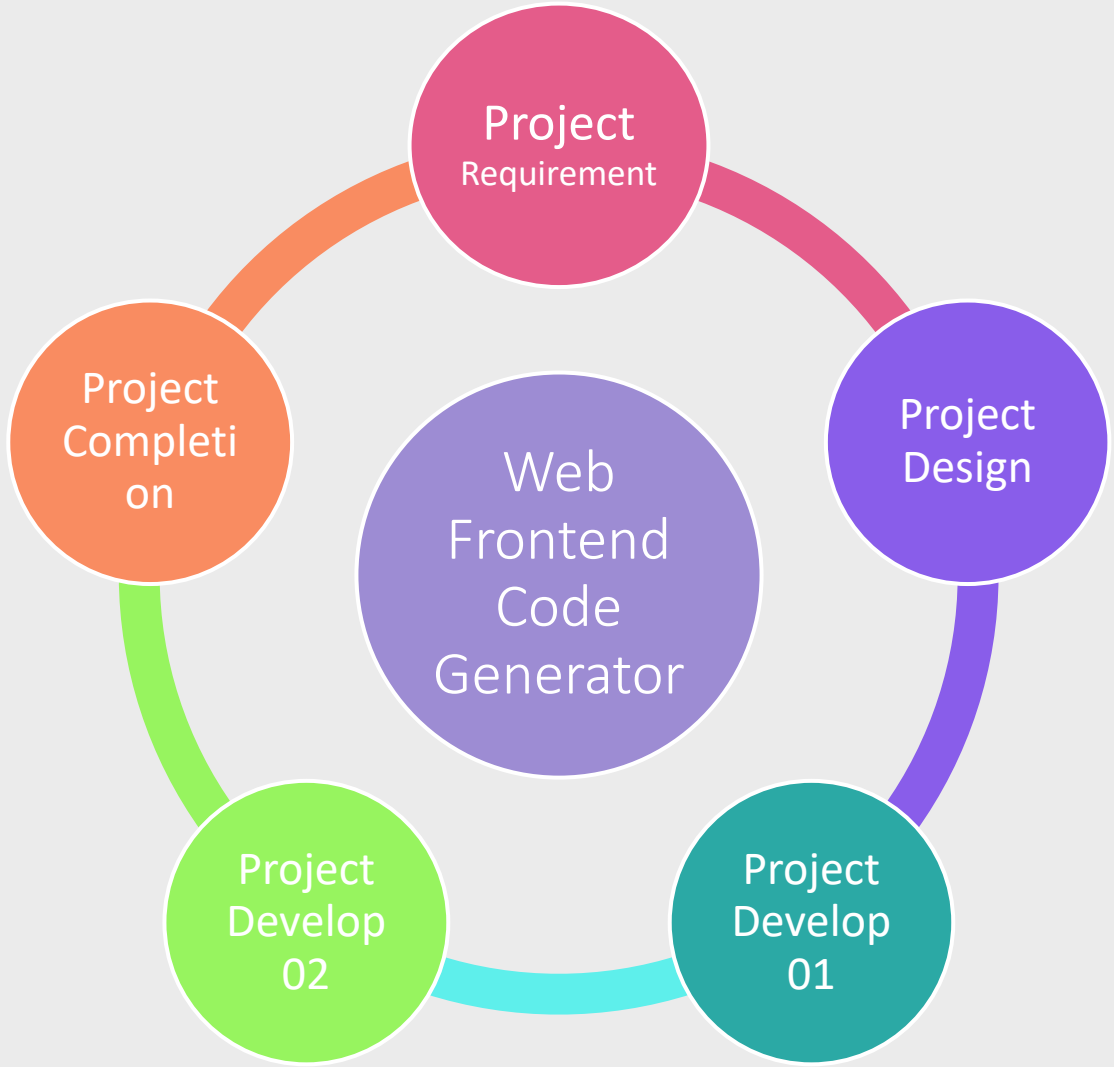
1. Flexible frontend implementation for user needs.
2. Performance & User-friendliness.
3. Coding Standards & Best Practices
4. Accuracy of generated code.



PROCESS


WEB FRONTEND CODE GENERATOR
UI GENERATOR
(COMPONENT 01)
XML
Original XML File
OUTPUT
XML
Sample XML File
INPUT
XML
json
XML to JSON
Conversion
pre-defined templates
Add Styles (CSS)
Web Frontend
Implementation
Display generated
code + UI result
Download generated
code
OUTPUT

PROJECT PHASE



CURRENT PROGRESS...



- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Initiated frontend development with UI navigation and styles.
- Completed converting XML to JSON.
- Created pre-define templates that can be selected according to user requirements.
- Generate and preview the sample code with UI.

REQUIREMENTS - FUNCTIONAL

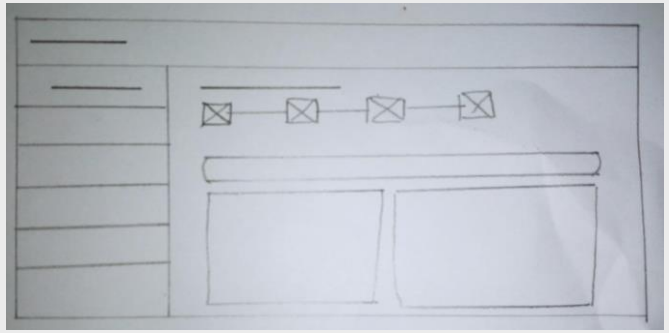
1. Browse/Drag sample XML file.
2. Convert XML to formatted JSON and submit.
3. Provide pre-define templates & Create style form for gather user requirements.
4. Generate frontend code for identified JSON details & submitted styles.
5. Display generated frontend code + UI result and download code.

"Continuous discussion with supervisor helped to sort requirements and finalize"



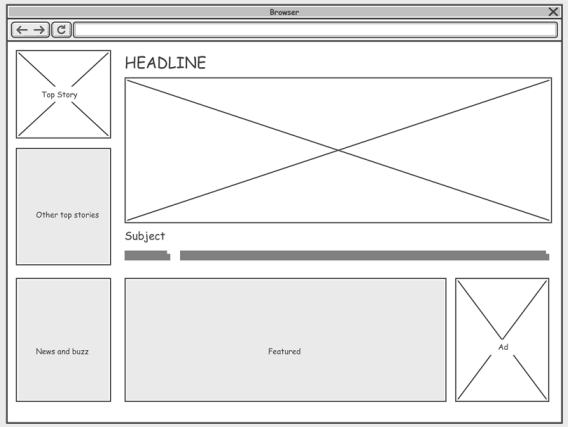
DESIGN.

Sketching



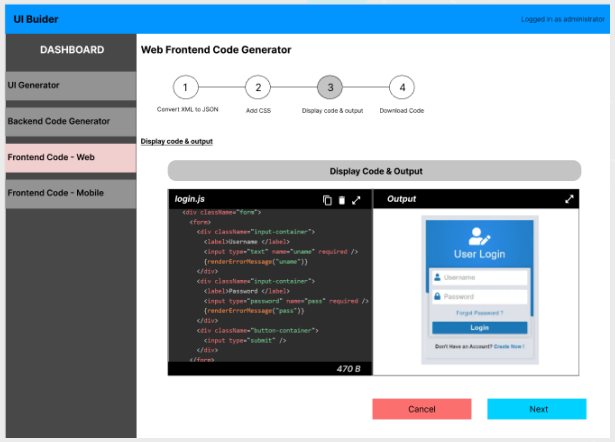
- Used pen & paper to sketch the frontend UI.
- Identified different components that is essential to implement features.
- Connecting UI navigations.

Wireframing



- Used Mock Flow to draw frontend wireframes.
- Visualize the component structure clearly.
- Clarify the layouts and alignments.

Prototyping



- Used Figma tool to create the prototypes.
- Identified best colors and styles for interfaces.
- The prototype served as the best evidence to show how the final output looks like.

IMPLEMENTATION – PHASE 01.



- Browse/Drag sample XML file and submit.
- Convert XML into JSON format and submit into project.
- Generate pre-define template styles.
- Create Style Form for gather user requirements.
- Generate a sample frontend code to facilitate the implementation phase-2 process.
- Display sample frontend code and UI result for user view.
- Complete sample frontend code download process.

KEY PILLARS



1. The approach of web frontend code generation.
2. Input and desired output.
3. Convert XML to JSON.
4. Pre-define template styles selection.

IMPLEMENTATION – PHASE 01 (EVIDENCES)

XML File

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:TestPro" x:Name="LoginScreen"
x:Class="TestPro.Window1" mc:Ignorable="d">
   <StackPanel Margin="300,150,293,83" Width="350">
            <Form x:Type="form">
                <Label x:type="header" Name="FormHeading" Content="Login
Form" FontSize="18" HorizontalAlignment="Center"/>
                <Separator x:type="horizontal-line" Margin="0,2,0,10"/>
                <Label x:type="label" Name="usernameTxt" Content="Username"
Margin="15,15,15,0"/>
                <TextBox x:type="input" x:Name="usrTextBox" Height="25"
Margin="15,0,15,0"/>
                <Label x:type="label" Name="passwordTxt" Content="Password"
Margin="15,15,15,0"/>
                <TextBox x:type="input" Name="pwdTextBox" Margin="15,0,15,40"
Height="25"/>
                <Button x:type="button" x:Name="submitBtm" Content="Login"
Width="150" Height="25"/>
            </Form>
   </StackPanel>
```

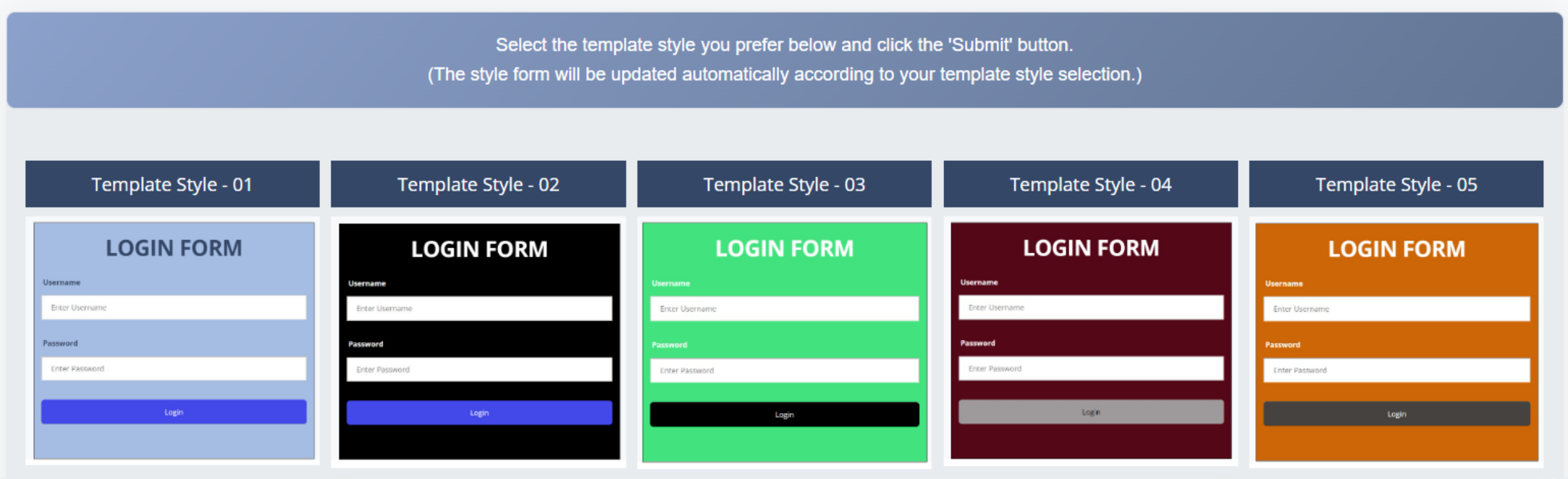
Sample Refine XML File

JSON File

```
{
 "Window": {
  "$": {
   "xmlns": "http://schemas.microsoft.com/winfx/2006/xaml/presentation",
   "xmlns:x": "http://schemas.microsoft.com/winfx/2006/xaml",
   "xmlns:d": "http://schemas.microsoft.com/expression/blend/2008",
   "xmlns:mc": "http://schemas.openxmlformats.org/markup-compatibility/2006",
   "xmlns:local": "clr-namespace:TestPro",
   "x:Name": "LoginScreen",
   "x:Class": "TestPro.Window1",
   "mc:Ignorable": "d"
  },
  "StackPanel": [
   {
    "$": {
     "Margin": "300,150,293,83",
     "Width": "350"
    },
    "Form": [
     {
      "$": {
       "x:Type": "form"
      },
```

Formatted JSON File

IMPLEMENTATION – PHASE 01 (EVIDENCES)



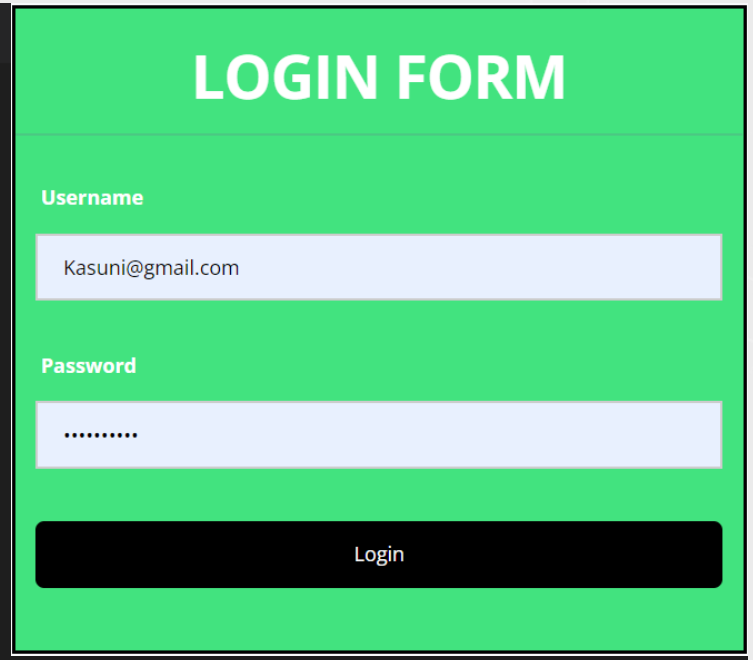
Pre-define Template Styles

IMPLEMENTATION – PHASE 01 (EVIDENCES)

Web_Frontend_Code.js

C: > Users > kasun > Downloads > Web_Frontend_Code.js > ...

```jsx
<div className="LoginForm">
    <form style={{ border: '3px solid #000000', backgroundColor: '#42e37f', width: '43%', marginLeft: '29%' }}>
        <div class="formContainer" style={{ textAlign: 'center', margin: '24px 0 12px 0' }}>
            <h1 style={{ color: 'white' }}>LOGIN FORM</h1><hr/>
        </div>
        <div class="container" style={{ padding: '16px' }}>
            <label for="uname" style={{ float: 'left', paddingTop: '5px', fontSize: '16px' }}>
                <b style={{ color: 'white' }}>Username</b>
            </label>
            <input
                type="text"
                placeholder="Enter Username"
                name="uname"
                required
                style={{ padding: '12px 20px', margin: '8px 0', border: '2px solid #ccc', width: '100%' }}
            />
            <br /><br />
            <label for="psw" style={{ float: 'left', paddingTop: '5px', fontSize: '16px' }}>
                <b style={{ color: 'white' }}>Password</b>
            </label>
            <input
                type="password"
                placeholder="Enter Password"
                name="psw"
                required
                style={{ padding: '12px 20px', margin: '8px 0', border: '2px solid #ccc', width: '100%' }}
            />
            <br /><br />
            <button type="submit" style={{ backgroundColor: '#000000', color: 'white', padding: '14px 20px', margin: '8px 0', border: 'none', borderRadius: '7px', width: '100%' }}>Login</button>
            <br /><br /><br />
        </div>
    </form>
</div>
```



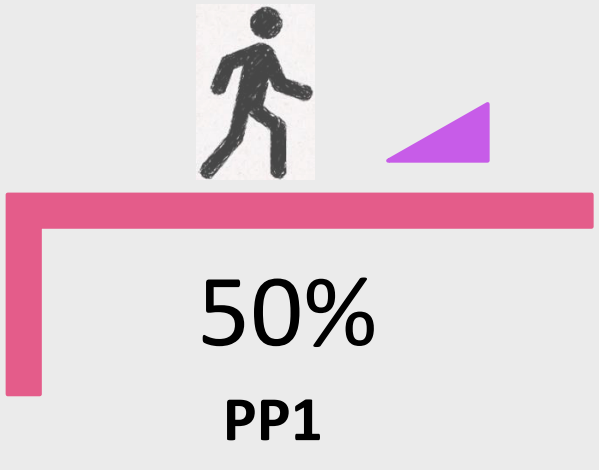
Sample Generated Frontend Code and result UI

IMPLEMENTATION – PHASE 02.



- Continue generating frontend implementation by appending JSON details and styles.
- Fix the newly generated frontend code instead of the sample React code process.
- Finishing UI/UX designs.
- Complete code refactoring and code commenting.

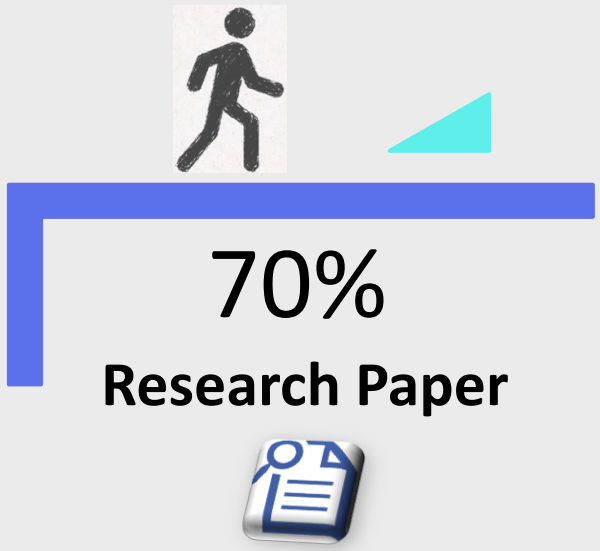
PROJECT COMPLETION STRATEGY



50%

PP1

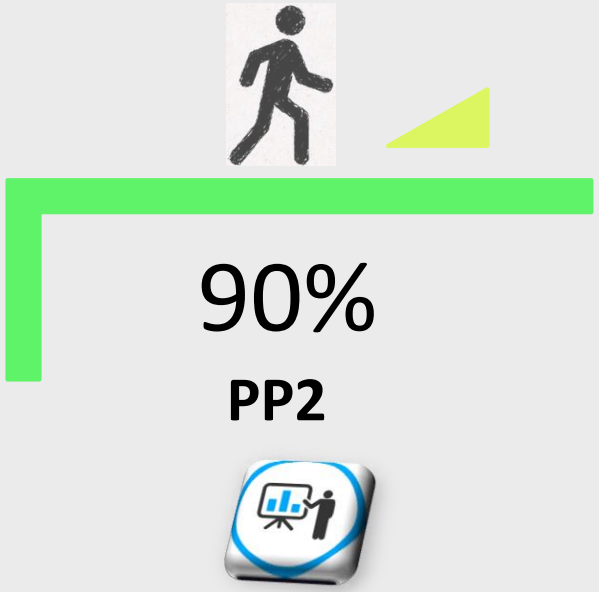
- Requirements & Design.
- Project Implementation – PHASE 01



70%

Research Paper

- Handle Frontend-Code UI/UX designs
- Handle validations and user flows
- Unit Testing



90%

PP2

- Completion of Project Implementation – PHASE 02
- Integrate with other components
- Integration Testing



100%

Final Presentation

- System Testing
- Project Completion



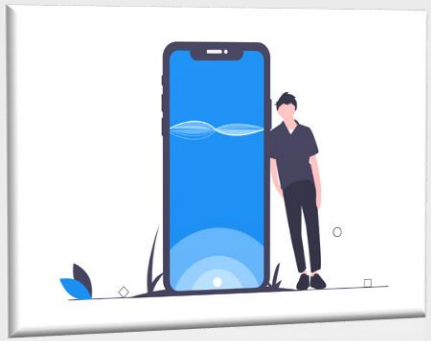
IT19150826|D.P. KAVINDI GIMSHANI

BSC (HONS) DEGREE IN INFORMATION TECHNOLOGY

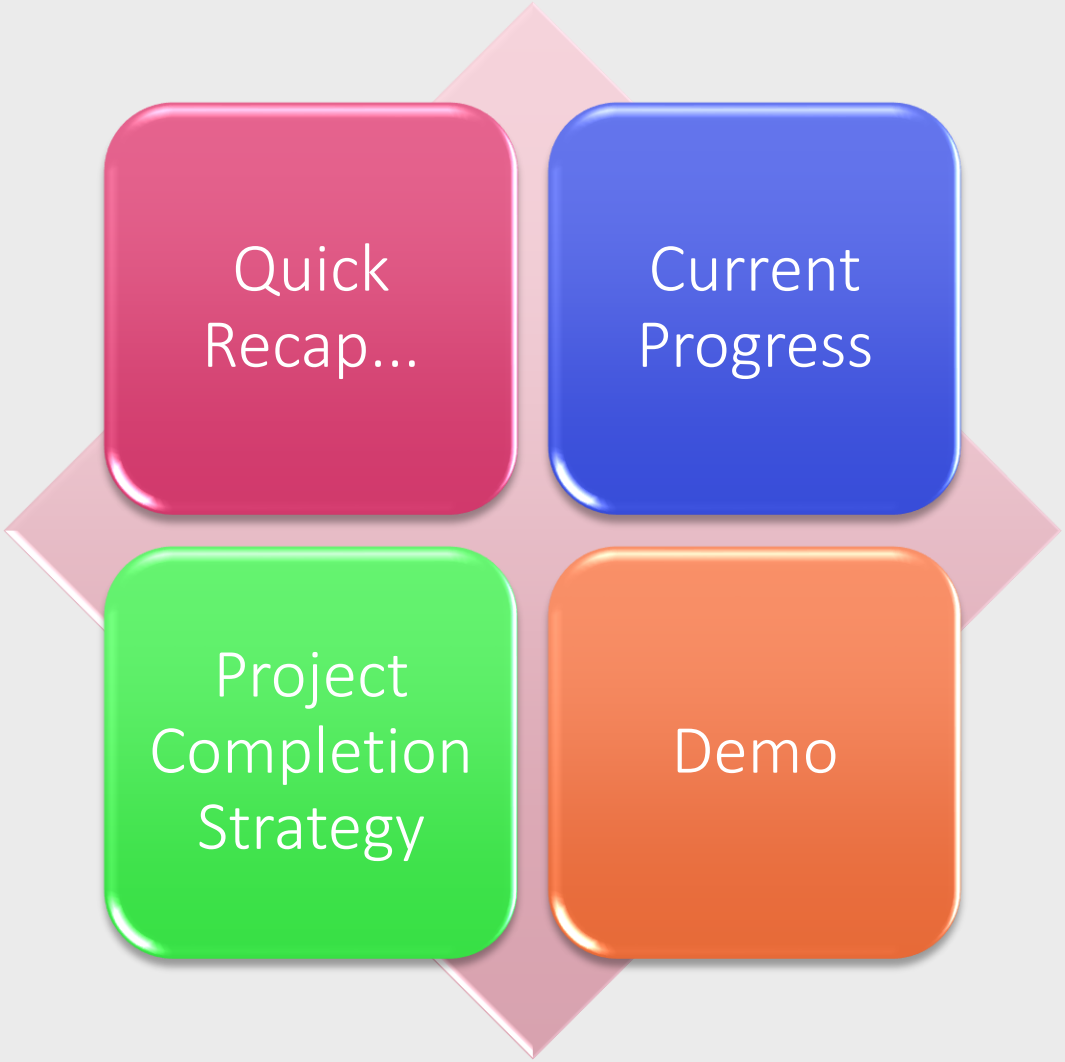
(SPECIALIZATION IN SOFTWARE ENGINEERING)

RESEARCH COMPONENT

FRONT-END CODE GENERATION FOR MOBILE APPLICATIONS USING CODE MAPPING



INTRODUCTION



QUICK RECAP.....

Research Question

How to generate the mobile frontend code for cross-platform applications?

1. How to manage the responsiveness of the code for multiple devices?
2. What are the challenges users go through while developing mobile applications?
3. What are the standards and other expectations from an auto-generated mobile code?



Objectives

Generate simple and accurate automated code for mobile applications while keeping the responsiveness.

1. User friendly code generation process.
2. Meet the expectations of the end users.
3. Allow the customization of the code for the end users.



QUICK RECAP.....

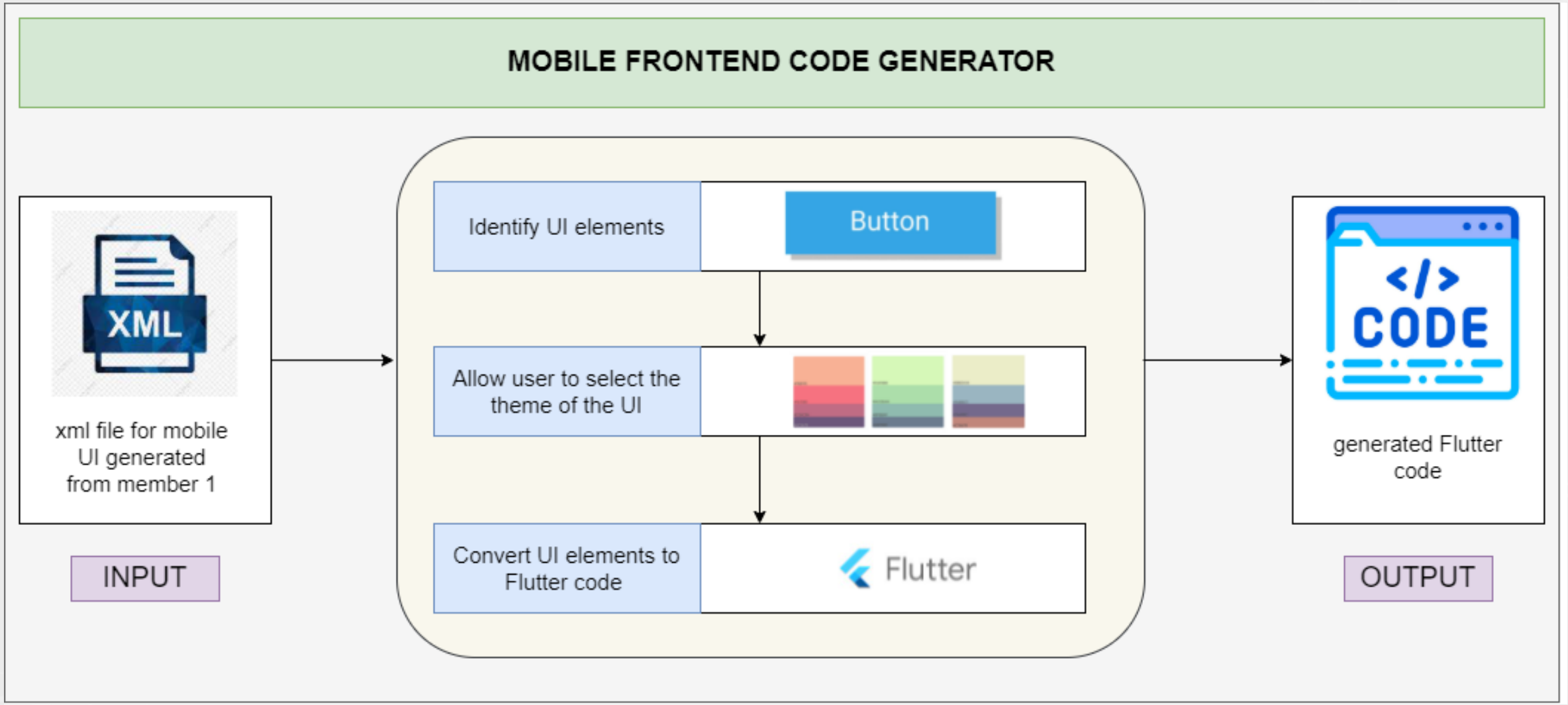
Research Gap

Single codebase for multi-platform mobile applications.

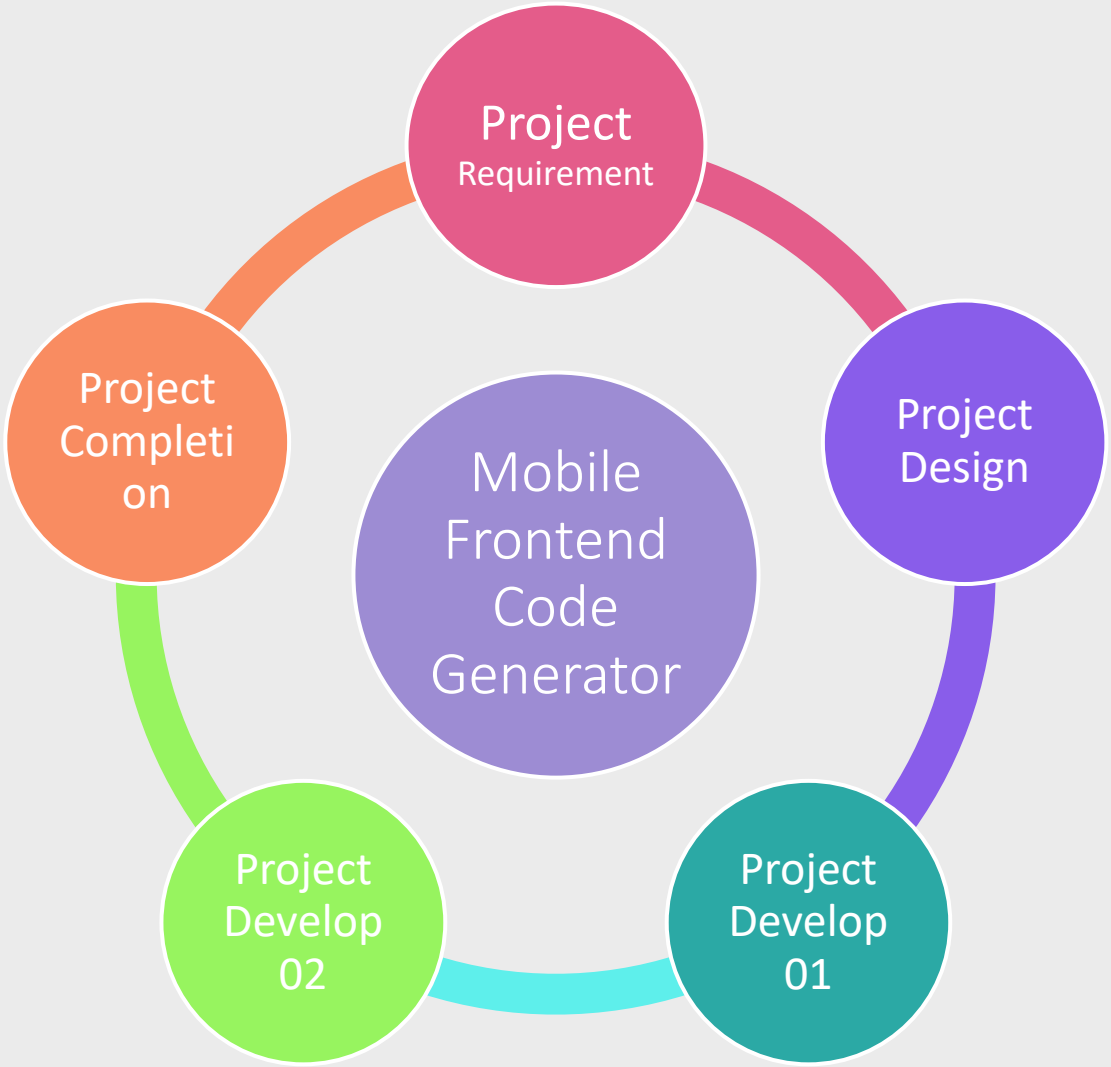
1. Suitable for cross-platform applications
2. Responsiveness
3. Flexible for the user requirements
4. Coding Standards & Best Practices(remove)



PROCESS


MOBILE FRONTEND CODE GENERATOR
XML
xml file for mobile UI generated from member 1
INPUT
Identify UI elements
Button
Allow user to select the theme of the UI
Convert UI elements to Flutter code
Flutter
CODE
generated Flutter code
OUTPUT

PROJECT PHASE



CURRENT PROGRESS...



- Completed identification of requirements and documentation.
- Sketching, wireframing and prototyping of interfaces.
- Initiated frontend development with UI navigation and styles.
- Retrieve data from the XML file and map them to relevant components.
- Allow user to select the pre-defined color theme for the interface.
- Convert the UI components into the Flutter code and write it to a file (Dart File).

REQUIREMENTS - FUNCTIONAL

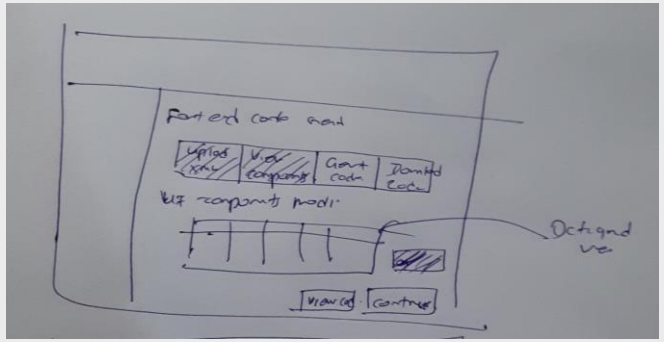
1. Upload the XML file to the system.
2. Read UI components in the XML file.
3. Let the user to select the color theme for the UI.
4. Generate the frontend code for the components and according to the color theme.
5. Display the generated code.

"Continuous discussion with supervisor helped to sort requirements and finalize"



DESIGN.

Sketching



- Used pen & paper to sketch the frontend UI.
- Identified different components th at is essential to implement features.
- Connecting UI navigations.

Wireframing

MockFlow

- Used Mock Flow to draw frontend wireframes.
- Visualize the component structure clearly.
- Clarify the layouts and alignments.

Prototyping

- Used Figma tool to create the prototypes.
- Identified best colors and styles for interfaces.
- The prototype served as the best evidence to show how the final output looks like.

IMPLEMENTATION – PHASE 01.



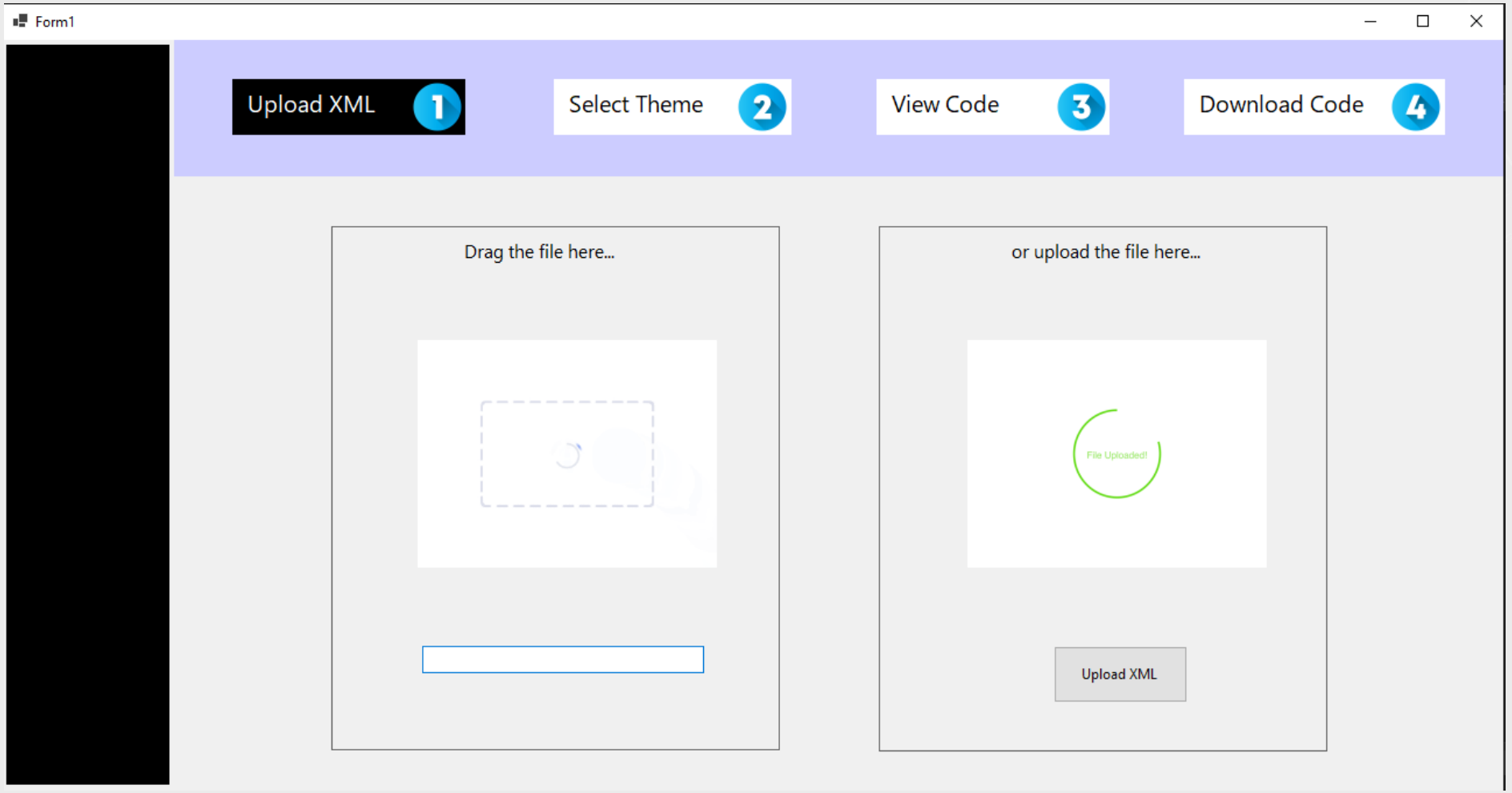
- Allow users to submit the XML file to the system.
- Read UI elements in XML file.
- Allow the user to select the color theme.
- Generate UI code according to the color theme.
- Display the generated code.

IMPLEMENTATION – PHASE 02.

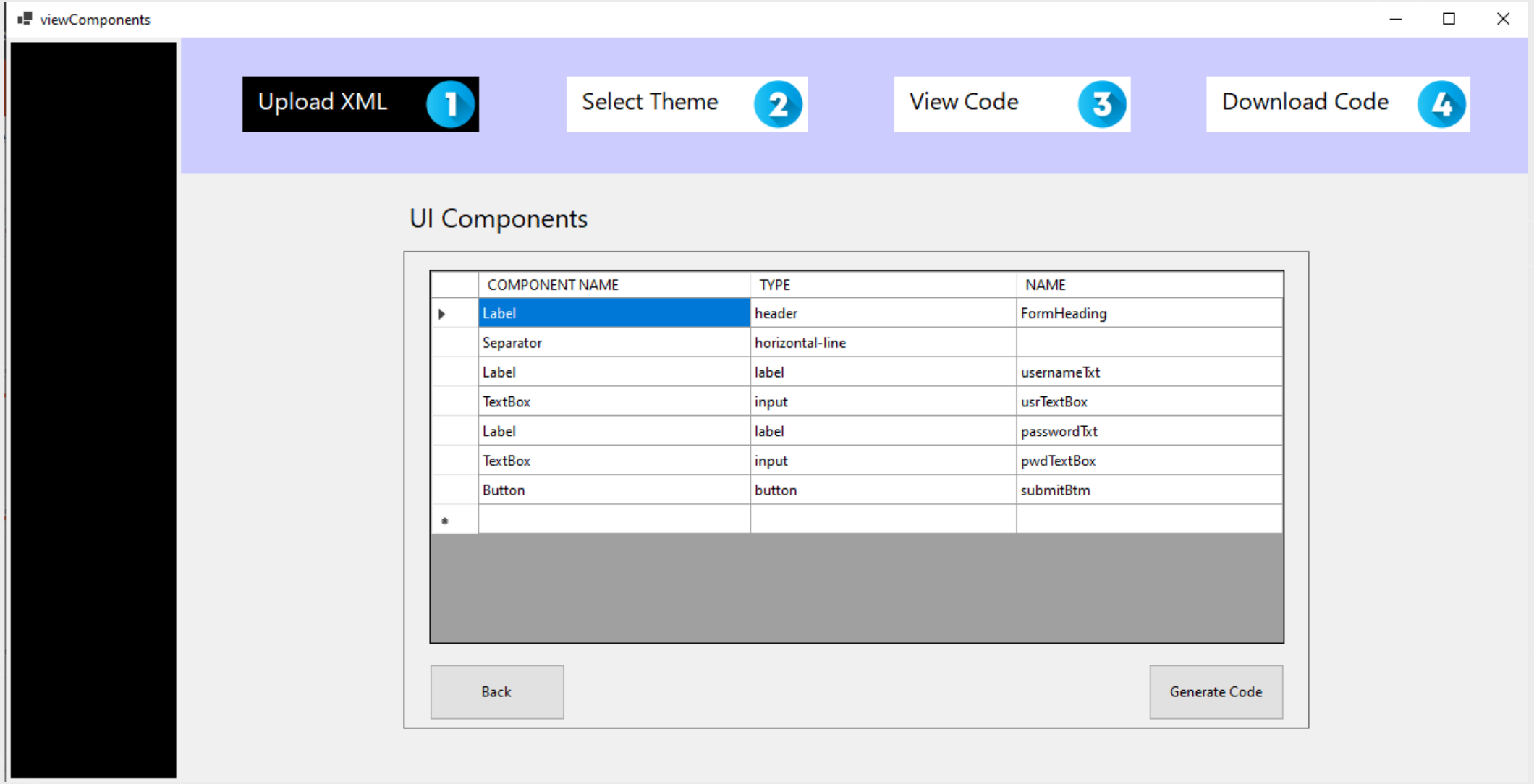


- Generate the code for more UI components
- Let the user to download the generated code
- Finish the interfaces of the system.

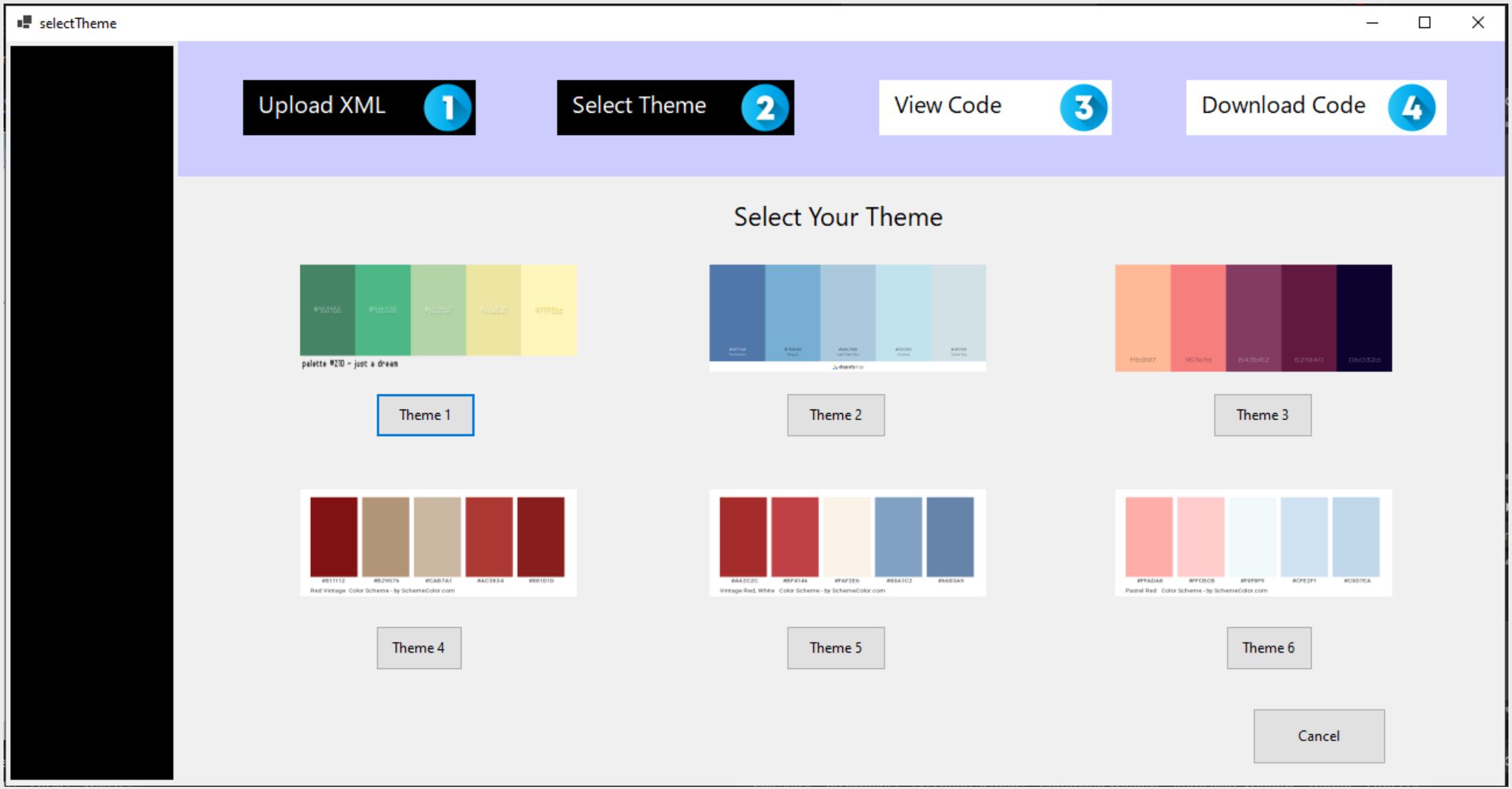
IMPLEMENTATION – PHASE 01 (EVIDENCES)



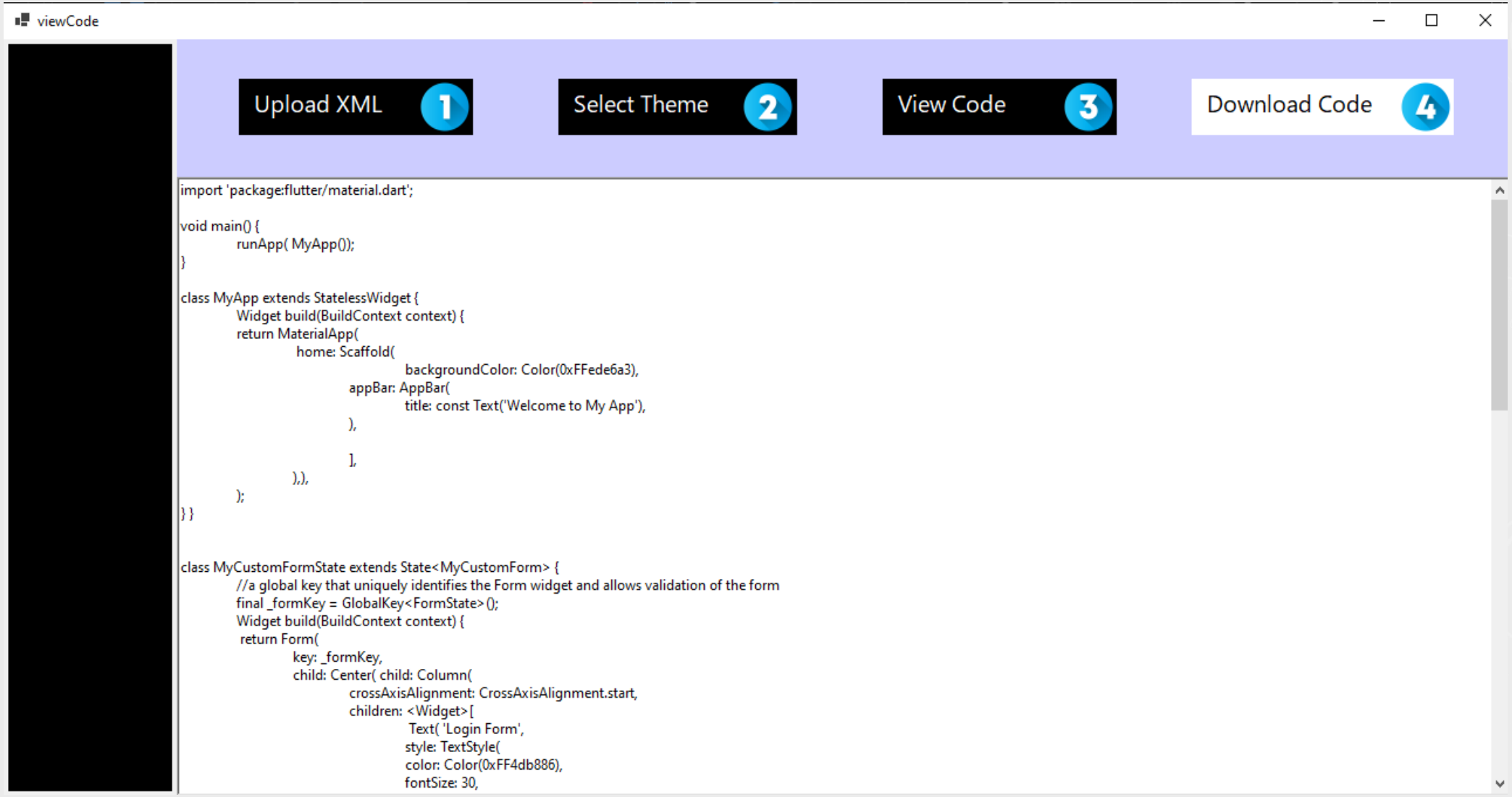
IMPLEMENTATION – PHASE 01 (EVIDENCES)



IMPLEMENTATION – PHASE 01 (EVIDENCES)



IMPLEMENTATION – PHASE 01 (EVIDENCES)



SAMPLE GENERATED CODE

WriteTextClone - Notepad

File Edit Format View Help

```
import 'package:flutter/material.dart';

void main() {
        runApp( MyApp());
}

class MyApp extends StatelessWidget {
        Widget build(BuildContext context) {
        return MaterialApp(
                 home: Scaffold(
                                backgroundColor: Color(0xFFede6a3),
                        appBar: AppBar(
                                title: const Text('Welcome to My App'),
                        ),
                        body: MyCustomForm(),
                        ),
        );
 }}

// Create a Form widget.
class MyCustomForm extends StatefulWidget{
        MyCustomFormState createState() {
                return MyCustomFormState();
        }
}

class MyCustomFormState extends State<MyCustomForm> {
        //a global key that uniquely identifies the Form widget and allows validation of
        final _formKey = GlobalKey<FormState>();
        Widget build(BuildContext context) {
         return Form(
                key: _formKey,
                child: Center( child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: <Widget>[
```

In application

```
import 'package:flutter/material.dart';

void main() {
  runApp( MyApp());
}

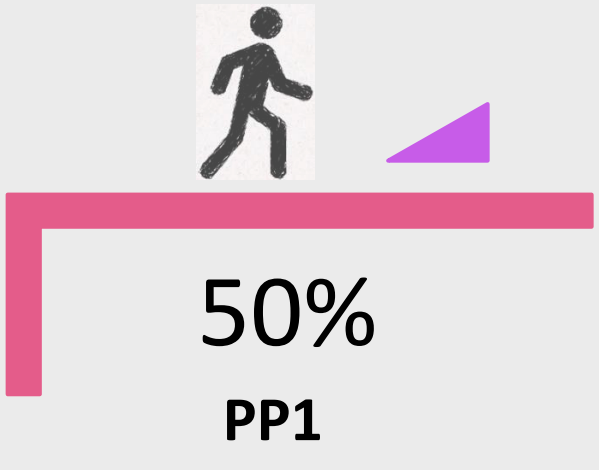
class MyApp extends StatelessWidget {
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        backgroundColor: Color(0xFFede6a3),
        appBar: AppBar(
          title: const Text('Welcome to My App'),
        ),
        body: MyCustomForm(),
      ),
    );
  }}

// Create a Form widget.
class MyCustomForm extends StatefulWidget{
  MyCustomFormState createState() {
    return MyCustomFormState();
  }
}

class MyCustomFormState extends State<MyCustomForm> {
```

In Android Studio

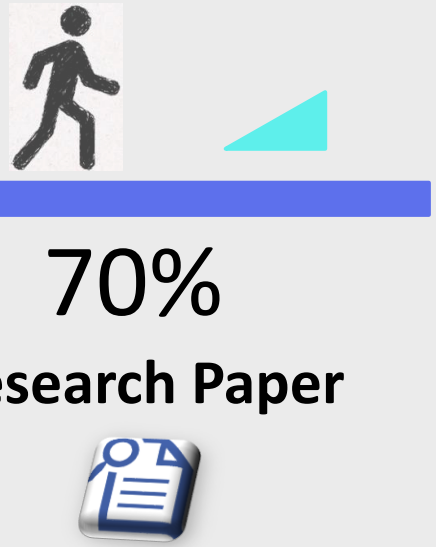
PROJECT COMPLETION STRATEGY



50%

PP1

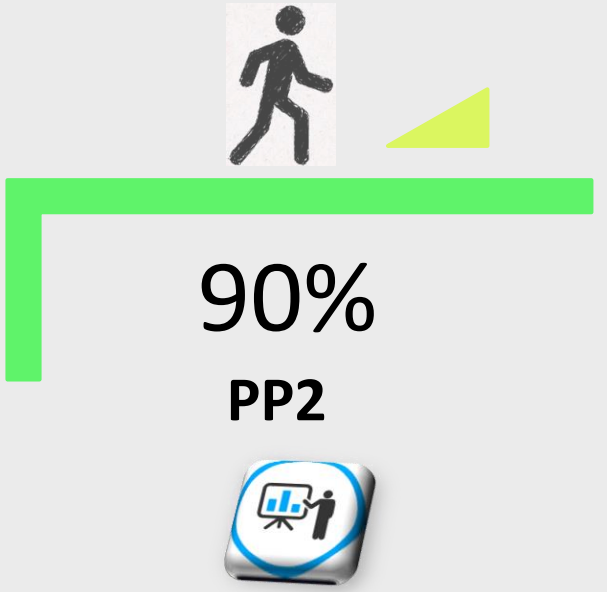
- Requirements & Design.
- Project Implementation – PHASE 01



70%

Research Paper

- Handle Frontend-Code UI/UX designs
- Handle validations and user flows
- Unit Testing



90%

PP2

- Completion of Project Implementation – PHASE 02
- Integrate with other components
- Integration Testing



100%

Final Presentation

- System Testing
- Project Completion

COMMERCIALIZATION

- Developers being the targeted users of our solution.
- Our tools helps developers from startups and organizations that involve in developing social software products to reduce their development time and enhance their process.
- The approach used in our tool is a single technology where both web and mobile development teams will be benefitted.
- Licensing schemes targeting individual software engineers, UI designers, frontend and backend developers.
- Separate flexible licensing schemes targeting software companies.
- Standard, professional and enterprise level versions of the solution available.



REFERENCES

[1]. Beltramelli, Tony. "pix2code: Generating code from a graphical user interface screenshot." Proceedings of the ACM SIGCHI Symposium on Engineering Interactive Computing Systems. 2018.

[2]. d'Amorim, Marcelo, Rui Abreu, and Carlos Mello. "Visual sketching: from image sketches to code." Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results. 2020.

[3]. Uyanik, Burak, and Veysel Harun Şahin. "A template-based code generator for web applications." Turkish Journal of Electrical Engineering & Computer Sciences 28.3 (2020): 1747-1762.

[4]. Aşıroğlu, Batuhan, et al. "Automatic HTML code generation from mock-up images using machine learning techniques." 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT). Ieee, 2019.

[5]. Beltramelli, Tony. (2017). pix2code: Generating Code from a Graphical User Interface Screenshot.

[6]. Deng, Youqiang & Fu, Cai & Li, Yang. (2019). CodeeGAN: Code Generation via Adversarial Training. 10.1007/978-981-15-1304-6_2.

[7]. B. Aşıroğlu et al., "Automatic HTML Code Generation from Mock-Up Images Using Machine Learning Techniques," 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT), 2019, pp. 1-4, doi: 10.1109/EBBT.2019.8741736.

[8]. M. d'Amorim, R. Abreu and C. Mello, "Visual Sketching: From Image Sketches to Code," 2020 IEEE/ACM 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER), 2020, pp. 101-104.

[9]. B. Vogel-Heuser, D. Witsch and U. Katzke, "Automatic code generation from a UML model to IEC 61131-3 and system configuration tools," 2005 International Conference on Control and Automation, 2005, pp. 1034-1039 Vol. 2, doi: 10.1109/ICCA.2005.1528274.

[10]. Jordi Cabot, “AI-based tools to transform interface design mockups into ready-to-use UI code.” https://modeling-languages.com/ai-tools-transform-design-mockups-into-ui-code/ [Accessed: 20 – Jan - 2022]

[11]. Noah Gundotra, "Code2Pix — Deep Learning Compiler for Graphical User Interfaces." https://towardsdatascience.com/code2pix-deep-learning-compiler-for-graphical-user-interfaces-1256c346950b [Accessed: 20 – Jan - 2022]

[12]. Vanita Jain, Piyush Agrawal, Subham Banga, Rishabh Kapoor, Shashwat Gulyani, "Sketch2Code: Transformation of Sketches to UI in Real-time Using Deep Neural Network." https://arxiv.org/abs/1910.08930 [Accessed: 20 – Jan - 2022]

[13]. M. Usman, A. Nadeem and T. -h. Kim, "UJECTOR: A Tool for Executable Code Generation from UML Models," 2008 Advanced Software Engineering and Its Applications, 2008, pp. 165-170, doi: 10.1109/ASEA.2008.39.

[14]. S. D. Rathod, "Automatic code generation with business logic by capturing attributes from user interface via XML," 2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT), 2016, pp. 1480-1484, doi: 10.1109/ICEEOT.2016.7754929.

[15]. IFML: The Interaction Flow Modeling Language: https://www.ifml.org/

[16]. Behrens, Heiko. "MDSD for the iPhone: developing a domain-specific language and IDE tooling to produce real world applications for mobile devices." *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*. 2010.

[17]. Parada, Abilio, Milena Marques, and Lisane B. de Brisolara. "Automating mobile application development: UML-based code generation for Android and Windows Phone." *Revista de Informática Teórica e Aplicada* 22.2 (2015): 31-50.



SLIIT FACULTY OF COMPUTING

Credits: This presentation was created by 2022 – 070 team,
initial template was given by SLIIT and reviewed by Dr. Nuwan Kodagoda.

NOW IT’S TIME FOR DEMO

